

**OPTIMISASI PERFORMA APLIKASI WEB SINGLE PAGE APPLICATION (SPA)
MENGUNAKAN HYBRID LAZY LOADING DAN CODE SPLITTING
PADA FRAMEWORK VUE.JS DENGAN VITE BUILD TOOL BERDASARKAN
TINGKAT KOMPLEKSITAS APLIKASI WEB**



MUSTARI MUHIDDIN

D082241013



**PROGRAM STUDI MAGISTER TEKNIK INFORMATIKA FAKULTAS TEKNIK
UNIVERSITAS HASANUDDIN GOWA**

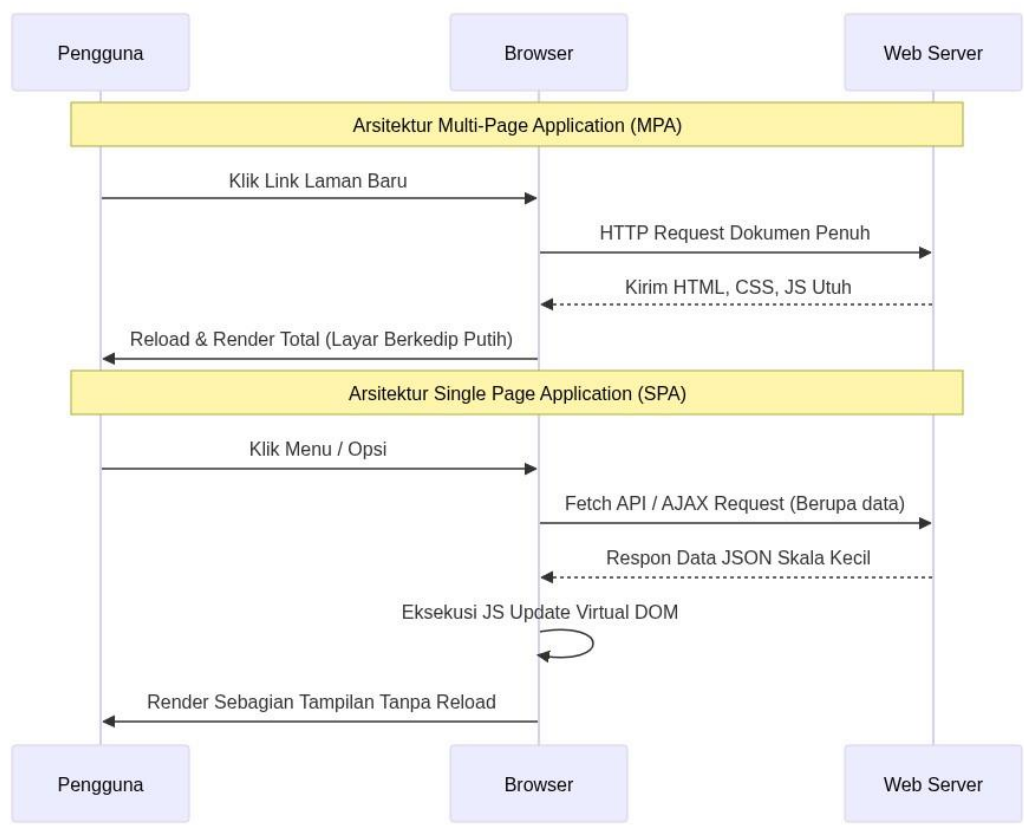
2026

BAB I PENDAHULUAN

1.1 Latar Belakang Masalah

Single Page Application (SPA) telah menjadi arsitektur dominan dalam pengembangan aplikasi web modern karena kemampuannya memberikan pengalaman pengguna yang responsif dan interaktif (Kowalczyk & Szandała, 2024). Berbeda dengan aplikasi web tradisional Multi-Page Application (MPA) yang memuat ulang seluruh halaman untuk setiap interaksi pengguna, SPA memuat seluruh konten dalam satu halaman HTML dan secara dinamis memperbarui tampilan tanpa reload halaman penuh, menghasilkan pengalaman yang lebih mirip aplikasi desktop (Taivalsaari & Mikkonen, 2021).

Dalam arsitektur MPA konvensional, setiap interaksi pengguna mengharuskan server memproses ulang halaman HTML secara utuh dan mengirimkannya kembali ke browser. Mekanisme pemuatan penuh (full page reload) ini tidak hanya membebani lalu lintas server, tetapi juga mendegradasi pengalaman pengguna karena timbulnya layar putih berkedip (white screen blanking) selama masa tunggu transmisi jaringan (Kowalczyk & Szandała, 2024).



Gambar 1.1 Perbandingan arsitektur MPA (Multi-Page Application) dan SPA (Single Page Application).

Gambar 1.1 di atas memperlihatkan perbedaan mendasar antara arsitektur MPA dan SPA. Pada MPA, setiap klik pengguna memaksa server untuk memproses dan mengirimkan ulang seluruh halaman HTML, menghasilkan layar putih berkedip yang

mengganggu pengalaman pengguna. Sebaliknya, pada SPA, server hanya mengirimkan data JSON berukuran kecil, sementara browser memperbarui tampilan secara parsial melalui JavaScript. Perbedaan inilah yang menjadi dasar mengapa ukuran bundle JavaScript pada SPA perlu dikelola dengan cermat melalui teknik seperti lazy loading dan code splitting.

Sebagai solusi dari masalah tersebut, paradigma SPA hadir sebagai standar industri mutakhir yang dipelopori oleh kerangka kerja JavaScript reaktif berbasis komponen seperti Vue.js, React, dan Angular. SPA memungkinkan aplikasi web untuk mengunduh satu kerangka HTML (index.html) pada muatan perdana, kemudian semua perubahan tampilan dikelola oleh JavaScript di browser. Ketika pengguna berpindah halaman, aplikasi hanya mengambil data kecil dalam format JSON dari server melalui API asinkron, lalu memperbarui bagian-bagian tertentu di layar saja (Taivalsaari & Mikkonen, 2021).

Namun, pertumbuhan kompleksitas aplikasi web menyebabkan peningkatan ukuran bundle JavaScript yang berdampak negatif pada performa. Ukuran bundle JavaScript yang besar berdampak langsung pada waktu muat halaman; keterlambatan pemuatan beberapa detik saja terbukti meningkatkan bounce rate secara signifikan (Google, 2020).

Ketika sebuah aplikasi SPA tidak dioptimalkan dengan benar, browser terpaksa mengunduh seluruh kode program — termasuk semua halaman, komponen, gambar, dan pustaka pihak ketiga — sekaligus dalam satu file JavaScript yang sangat besar (monolithic build). Untuk website sederhana seperti halaman profil perusahaan, hal ini mungkin tidak masalah. Tetapi untuk aplikasi yang kompleks seperti Sistem Informasi Manajemen Tugas Akhir (SIMTA) — yang menggunakan banyak fitur seperti grafik interaktif (Chart.js), manajemen data global (Pinia), koneksi database (Supabase), dan autentikasi pengguna — ukuran file-nya dapat melebihi batas wajar yang direkomendasikan untuk browser (lebih dari 300 KB terkompresi).

Pada tahap awal, aplikasi berkompleksitas tinggi ditetapkan melalui kajian awal (preliminary study). Kandidat awal berupa dashboard admin toko dinilai kurang representatif karena tidak memiliki ketergantungan pustaka eksternal yang cukup berat untuk menghasilkan perbedaan yang terukur secara signifikan pada strategi code splitting. Oleh karena itu, objek penelitian ditetapkan pada Sistem Informasi Manajemen Tugas Akhir (SIMTA) yang secara bersamaan menggunakan Chart.js (grafik interaktif), Pinia (state management), Supabase (koneksi database real-time), dan Vue Router dengan 15+ rute. Kombinasi ini menghasilkan bundle monolitik yang jauh lebih besar sehingga mewakili aplikasi kompleksitas tinggi secara lebih representatif, tanpa mengubah pertanyaan penelitian, variabel, maupun tujuan yang telah ditetapkan.

Akibat dari bundle yang terlalu besar ini sangat merugikan. Nilai-nilai penting dalam pengukuran performa web yang disebut Core Web Vitals — yang digunakan oleh mesin pencari seperti Google — akan turun drastis (Google Chrome Developers, 2023). Perbaikan pada metrik pemuatan seperti LCP dan FCP berkorelasi dengan peningkatan keterlibatan pengguna dan penurunan bounce rate. Saat browser harus memproses file

JavaScript yang sangat besar, seluruh kemampuan prosesor digunakan untuk membaca, menguraikan, dan menjalankan kode tersebut. Selama proses ini berlangsung, browser tidak bisa merespons klik atau interaksi pengguna sama sekali — kondisi yang diukur melalui metrik Total Blocking Time (TBT) (Google Chrome Developers, 2023).

Untuk mengatasi masalah ini, pendekatan Code Splitting dan Lazy Loading telah dikembangkan (Kumar, Singh & Sharma, 2024). Alih-alih mengirimkan semua kode sekaligus, hanya bagian yang diperlukan saat pengguna membuka halaman tertentu yang dikirimkan. Kumar, Singh, dan Sharma (2024) menunjukkan bahwa kombinasi lazy loading dan code splitting dapat menurunkan waktu muat halaman hingga sekitar 40% dibandingkan implementasi standar. Lebih lanjut, teknik Hybrid Lazy Loading atau Prefetching memanfaatkan waktu senggang browser untuk diam-diam mengunduh terlebih dahulu bagian kode yang kemungkinan akan dibutuhkan selanjutnya (Google Chrome Developers, 2023).

Tingkat kompleksitas aplikasi web mempengaruhi efektivitas strategi optimasi. Aplikasi dengan kompleksitas rendah seperti company profile memiliki karakteristik berbeda dengan aplikasi kompleksitas tinggi yang melibatkan operasi CRUD dan visualisasi data real-time. Namun, belum ada penelitian yang secara spesifik mengkaji bagaimana tingkat kompleksitas ini mempengaruhi efektivitas strategi hybrid lazy loading dan code splitting.

Berdasarkan permasalahan di atas, penelitian ini membahas topik: "Optimasi Performa Single Page Application Menggunakan Hybrid Lazy Loading dan Code Splitting Berdasarkan Tingkat Kompleksitas Sistem".

1.2 Rumusan Masalah

Berdasarkan permasalahan yang telah diuraikan, penelitian ini difokuskan pada pertanyaan-pertanyaan berikut:

1. Bagaimana pengaruh implementasi hybrid lazy loading dan code splitting terhadap metrik performa SPA (FCP, LCP, TBT, dan bundle size) pada aplikasi dengan tingkat kompleksitas tinggi (SIMTA)?
2. Bagaimana pengaruh implementasi hybrid lazy loading dan code splitting terhadap metrik performa SPA pada aplikasi dengan tingkat kompleksitas rendah (Company Profile)?
3. Bagaimana perbedaan efektivitas strategi hybrid lazy loading dan code splitting antara aplikasi kompleksitas tinggi dan rendah?
4. Bagaimana rekomendasi strategi optimasi hybrid lazy loading dan code splitting yang sesuai berdasarkan tingkat kompleksitas aplikasi web?

1.3 Batasan Masalah

Agar penelitian ini fokus dan hasilnya dapat diukur dengan jelas, batasan penelitian ditetapkan sebagai berikut:

1. Framework dan Build Tool: Vue.js versi 3.x dengan Composition API, menggunakan Vite sebagai build tool.
2. Penelitian menggunakan dua jenis aplikasi sebagai objek percobaan: (a) Aplikasi Kompleks/Tinggi (SIMTA): Sistem Informasi Manajemen Tugas Akhir yang menggunakan Vue.js 3, Chart.js, Supabase, dan Pinia secara bersamaan; (b) Aplikasi Sederhana/Rendah (Company Profile): Website profil perusahaan yang hanya menampilkan konten statis tanpa grafik interaktif atau manajemen data yang kompleks.
3. Teknik Optimasi: Route-based lazy loading, component-based lazy loading, dynamic import, code splitting berbasis chunk, dan intelligent prefetching.
4. Metrik Performa: First Contentful Paint (FCP), Largest Contentful Paint (LCP), Total Blocking Time (TBT), Load Time, penggunaan memori JavaScript (JS Heap), serta data tambahan dari Lighthouse (Performance Score, Time to Interactive).
5. Alat pengukuran: Kombinasi W3C PerformanceObserver (pengukuran langsung dari browser) dan Google Lighthouse (sebagai alat audit standar industri), dijalankan secara otomatis melalui Puppeteer.
6. Pengujian: Dilakukan di server lokal (localhost) menggunakan Puppeteer untuk automasi browser, dengan simulasi pelambatan CPU 4x melalui Puppeteer Chromium API, sehingga variabel jaringan dapat dieleminasi untuk mendapatkan pengukuran CPU-bound yang murni.
7. Tidak mencakup: Server-Side Rendering (SSR), Progressive Web App (PWA), dan optimasi backend/database.

1.4 Tujuan Penelitian

Penelitian ini bertujuan untuk:

1. Menganalisis pengaruh implementasi hybrid lazy loading dan code splitting terhadap metrik performa SPA pada aplikasi kompleksitas tinggi (SIMTA), khususnya dalam memisahkan pustaka-pustaka besar seperti Chart.js dan Pinia ke dalam file-file terpisah.
2. Menganalisis pengaruh implementasi hybrid lazy loading dan code splitting terhadap metrik performa SPA pada aplikasi kompleksitas rendah (Company Profile).
3. Membandingkan efektivitas strategi hybrid lazy loading dan code splitting antara aplikasi kompleksitas tinggi dan rendah.
4. Merumuskan rekomendasi strategi optimasi hybrid lazy loading dan code splitting berdasarkan tingkat kompleksitas aplikasi web.

1.5 Manfaat Penelitian

1.5.1 Manfaat Teoritis

- 1. Memberikan kontribusi pada body of knowledge terkait optimasi performa Single Page Application khususnya dalam konteks Vue.js dan Vite build tool.
- 2. Memperkaya literatur tentang strategi adaptive optimization yang mempertimbangkan karakteristik dan kompleksitas aplikasi web.
- 3. Memberikan referensi baru tentang cara mengoptimalkan performa SPA menggunakan pengukuran yang akurat dan langsung dari browser (W3C PerformanceObserver), sehingga hasilnya lebih representatif terhadap kondisi nyata.

1.5.2 Manfaat Praktis

- 1. Bagi Developer: Memberikan panduan praktis dalam menerapkan strategi hybrid lazy loading dan code splitting pada proyek Vue.js berdasarkan tingkat kompleksitas aplikasi.
- 2. Bagi Industri: Membantu organisasi dalam mengoptimalkan performa aplikasi web yang berdampak pada peningkatan user experience dan conversion rate.
- 3. Bagi Peneliti Selanjutnya: Menyediakan baseline dan metodologi yang dapat dikembangkan untuk penelitian lebih lanjut terkait optimasi performa web.

1.6 Tinjauan Pustaka / Penelitian Terdahulu

Optimasi performa Single Page Application melalui code splitting dan lazy loading telah menjadi fokus berbagai penelitian dalam beberapa tahun terakhir. Untuk memposisikan kontribusi penelitian ini secara jelas, Tabel 1.1 merangkum enam penelitian terdahulu yang paling relevan — mencakup fokus dan metode, temuan utama, serta kesenjangan (gap) masing-masing terhadap penelitian yang dilakukan. Sintesis dari keseluruhan tinjauan pustaka disajikan setelah tabel.

Tabel 1.1 Rangkuman Perbandingan Penelitian Terdahulu

No	Peneliti (Tahun)	Fokus & Metode	Hasil / Temuan	Gap & Perbedaan dengan Penelitian Ini
1	Kowalczyk & Szandala (2024)	Evaluasi SEO & visibilitas SPA vs MPA (IEEE Access); studi review + eksperimen.	SPA menghadapi tantangan indexing/SEO dibanding MPA; diusulkan strategi peningkatan visibilitas.	Fokus pada SEO, bukan optimasi performa. Penelitian ini fokus lazy loading + code splitting dan tingkat kompleksitas.
2	Emmanni (2023)	Analisis komparatif Angular, React, dan Vue.js untuk pengembangan SPA (benchmark + survei).	Setiap framework unggul pada konteks berbeda; pemilihan bergantung kebutuhan proyek.	Perbandingan bersifat umum; tidak menerapkan/mengukur teknik optimasi maupun faktor kompleksitas. Penelitian ini fokus Vue.js + Vite.

3	Bara, Boiangiu & Tudose (2024)	Analisis empiris dampak lazy loading (situs statis vs dinamis, variasi kondisi jaringan).	Lazy loading memperbaiki FCP/LCP, terutama pada jaringan lambat; terdapat trade-off eager vs lazy.	Hanya lazy loading tanpa code splitting; tidak spesifik Vue+Vite dan tidak mengklasifikasi tingkat kompleksitas.
4	Kumar, Singh & Sharma (2024)	Implementasi lazy loading + code splitting lintas framework (React, Angular, Vue).	Kombinasi keduanya menurunkan page load time hingga $\pm 40\%$.	Bersifat umum; tidak deep-dive Vue.js + Vite dan tidak mempertimbangkan tingkat kompleksitas sistem sebagai penentu strategi.
5	Setiawan & Fauzi (2025)	Komparatif lazy loading + code splitting pada React/Vue/Angular berdasarkan skor Lighthouse (FCP, LCP, TBT, CLS).	React waktu muat tercepat & stabilitas layout terbaik; Vue fleksibel; Angular lebih rendah.	Pengukuran hanya via Lighthouse (bukan PerformanceObserver real-user), tanpa uji tingkat kompleksitas & CPU throttling; keduanya ditambahkan di penelitian ini.
6	Taivalsaari & Mikkonen (2021)	Roadmap arsitektur SPA di era programmable world (konseptual).	SPA memberi pengalaman responsif; menyoroti tantangan arsitektur & beban di sisi klien.	Bersifat visioner/konseptual, bukan evaluasi teknik optimasi konkret.

Di luar keenam studi utama pada Tabel 1.1, sejumlah penelitian lain memperkuat konteks penelitian ini. Perbandingan performa antar-framework JavaScript modern telah banyak dikaji (Piastou, 2023; Jihadi & Syarabil, 2023; Sofi'e & Qoiriah, 2023; Anggraeni dkk., 2024; Khoirurrizal dkk., 2024; Wijaya & Farisi, 2025), termasuk pengembangan SPA berbasis React (Jonathan & Suprihadi, 2023) dan efisiensi build tool modern seperti Vite (Fauzi, 2024). Teknik lazy loading dan code splitting untuk meningkatkan responsivitas juga menjadi perhatian (Turcotte, Gokhale & Tip, 2023; Larissa & Suartana, 2026), demikian pula aspek pendukung seperti manajemen state (Donvir dkk., 2024), kualitas kode JavaScript dan TypeScript (Saboury dkk., 2017; Johannes dkk., 2019; Bogner & Merkel, 2022), mekanisme rendering dan Server-Side Rendering (Noer & Suartana, 2024; Hermanto & Engel, 2025), pengujian end-to-end (Rezeki dkk., 2026), serta tinjauan umum teknik optimasi performa web (Vepsäläinen, Hellas & Vuorimaa, 2023).

Gap Penelitian: Berdasarkan tinjauan di atas, belum terdapat penelitian komprehensif yang mengkaji implementasi hybrid lazy loading dan code splitting pada Vue.js dengan Vite yang mempertimbangkan tingkat kompleksitas aplikasi sebagai faktor penentu strategi optimasi. Penelitian ini mengisi gap tersebut dengan

membandingkan dua aplikasi dengan kompleksitas yang sangat berbeda, menggunakan kombinasi alat pengukuran (PerformanceObserver + Lighthouse), dan menguji pada kondisi perangkat yang terbatas (CPU throttling).

1.7 Sistematika Penulisan

Penulisan tesis ini dibagi menjadi empat bab utama:

- **BAB I PENDAHULUAN:** Menjelaskan latar belakang mengapa performa SPA perlu dioptimalkan, masalah yang ingin diselesaikan, batasan penelitian, tujuan, manfaat, dan ringkasan penelitian-penelitian terdahulu yang relevan.
- **BAB II METODE PENELITIAN DAN LANDASAN TEORI:** Menjelaskan teori-teori dasar yang digunakan serta metodologi penelitian secara detail termasuk variabel penelitian, spesifikasi alat yang digunakan, dan langkah-langkah pengujian.
- **BAB III HASIL DAN PEMBAHASAN:** Menyajikan hasil pengukuran dari kedua versi aplikasi (standar dan yang dioptimalkan), termasuk kode program, grafik perbandingan, data Lighthouse, analisis statistik, dan interpretasi mendalam tentang setiap temuan.
- **BAB IV PENUTUP:** Merangkum kesimpulan dari penelitian dan memberikan saran untuk penelitian lanjutan.

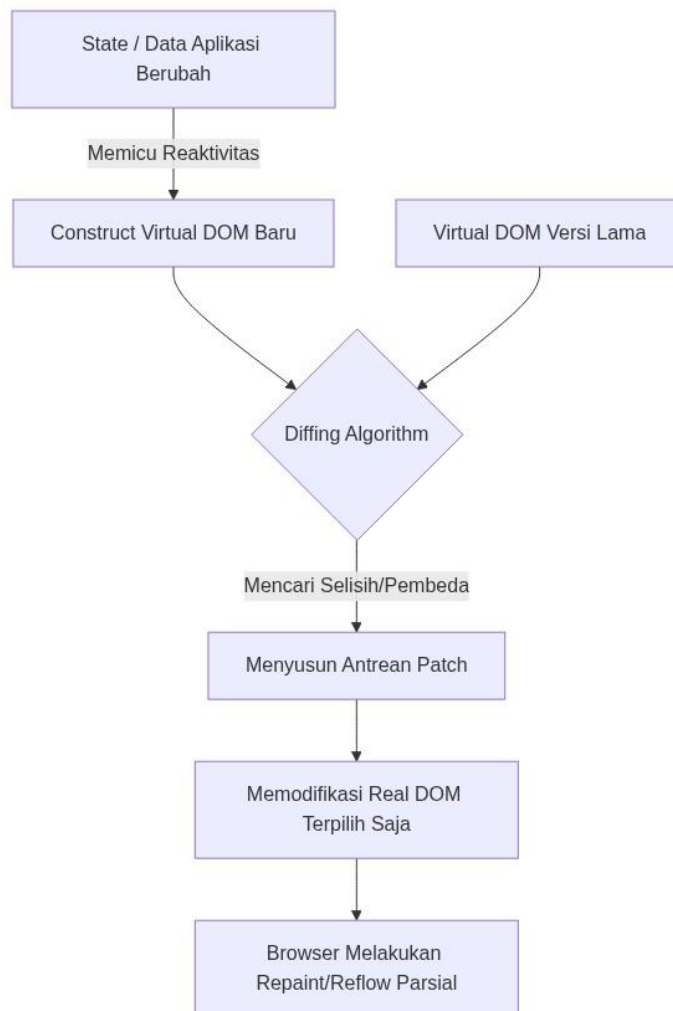
BAB II METODE PENELITIAN DAN LANDASAN TEORI

2.1 Landasan Teori

2.1.1 Arsitektur Single Page Application (SPA) dan Virtual DOM

Single Page Application (SPA) adalah aplikasi web yang berinteraksi dengan pengguna melalui pemuatan ulang dinamis halaman tunggal, berbeda dengan aplikasi web tradisional yang memuat ulang seluruh halaman untuk setiap interaksi (Kowalczyk & Szandala, 2024). Menurut Taivalsaari dan Mikkonen (2021), SPA menawarkan pengalaman pengguna yang lebih responsif karena hanya memperbarui konten yang diperlukan tanpa reload seluruh halaman.

Salah satu fitur unggulan SPA adalah penggunaan Virtual DOM. Virtual DOM adalah representasi maya dari tampilan halaman yang disimpan di memori JavaScript. Ketika ada data yang berubah, Vue.js terlebih dahulu membuat Virtual DOM baru, membandingkannya dengan salinan lama (diffing algorithm), lalu memperbarui tampilan nyata hanya pada bagian yang berbeda saja. Cara ini jauh lebih efisien dibandingkan memuat ulang seluruh halaman (You et al., 2023).



Gambar 2.1 Mekanisme pembaruan antarmuka melalui algoritma diffing Virtual DOM.

Gambar 2.1 di atas mengilustrasikan cara Virtual DOM mengoptimalkan proses pembaruan antarmuka. Daripada menggambar ulang seluruh halaman setiap kali data berubah, Vue.js hanya memperbarui node-node DOM yang benar-benar berbeda antara snapshot Virtual DOM lama dan baru. Efisiensi ini sangat relevan dengan topik penelitian karena menjelaskan mengapa penundaan parsing JavaScript melalui lazy loading tidak mengurangi kecepatan pembaruan antarmuka setelah komponen berhasil dimuat.

2.1.2 Vue.js Framework

Vue.js adalah progressive JavaScript framework yang dirancang untuk membangun user interface dengan pendekatan bottom-up incremental adoption (You et al., 2023). Vue.js 3 memperkenalkan Composition API sebagai alternatif Options API, serta reactivity system menggunakan ES6 Proxy yang menghasilkan performa lebih baik. Menurut dokumentasi resmi Vue.js (You et al., 2023), Vue 3 menghadirkan peningkatan rendering performance, memory footprint yang lebih rendah, serta bundle size yang lebih kecil dibandingkan Vue 2.

2.1.3 Vite Build Tool dan Code Splitting

Vite adalah build tool modern yang dikembangkan oleh Evan You dengan fokus pada developer experience dan performa (Vite Team, 2024). Berbeda dengan bundler tradisional seperti Webpack, Vite memanfaatkan native ES modules di browser untuk melayani kode secara on-demand. Vite juga menyediakan optimasi ukuran bundle secara default melalui konfigurasi Rollup yang telah dioptimasi untuk web (Vite Team, 2024). Untuk production build, Vite menggunakan Rollup sebagai bundler dengan konfigurasi yang sudah dioptimasi, termasuk code splitting berbasis dynamic import dan vendor chunk separation.

2.1.4 Lazy Loading

Lazy loading adalah teknik optimasi yang menunda loading resource hingga benar-benar diperlukan oleh pengguna (Bara, Boiangiu & Tudose, 2024). Terdapat beberapa strategi yang dapat diterapkan pada Vue.js:

- **Route-based Lazy Loading:** Memuat komponen route hanya ketika route tersebut diakses pertama kali.
- **Component-based Lazy Loading:** Memuat komponen individual secara on-demand, biasanya untuk komponen yang berat atau jarang digunakan.
- **Conditional Lazy Loading:** Memuat komponen berdasarkan kondisi tertentu seperti user role atau device type.

Bara, Boiangiu, dan Tudose (2024) menemukan bahwa penerapan lazy loading efektif mengurangi initial bundle size dan memperbaiki metrik pemuatan awal, terutama pada kondisi jaringan lambat.

2.1.5 Strategi Optimasi Hibrida

Penelitian terbaru menunjukkan bahwa pendekatan hibrida yang mengkombinasikan beberapa teknik optimasi menghasilkan hasil lebih baik dibandingkan pendekatan tunggal (Setiawan & Fauzi, 2025). Efektivitas strategi hibrida ini bergantung pada karakteristik aplikasi, termasuk jumlah routes dan komponen, ukuran individual komponen, kompleksitas dependency graph, pola navigasi pengguna, serta kondisi target perangkat dan jaringan.

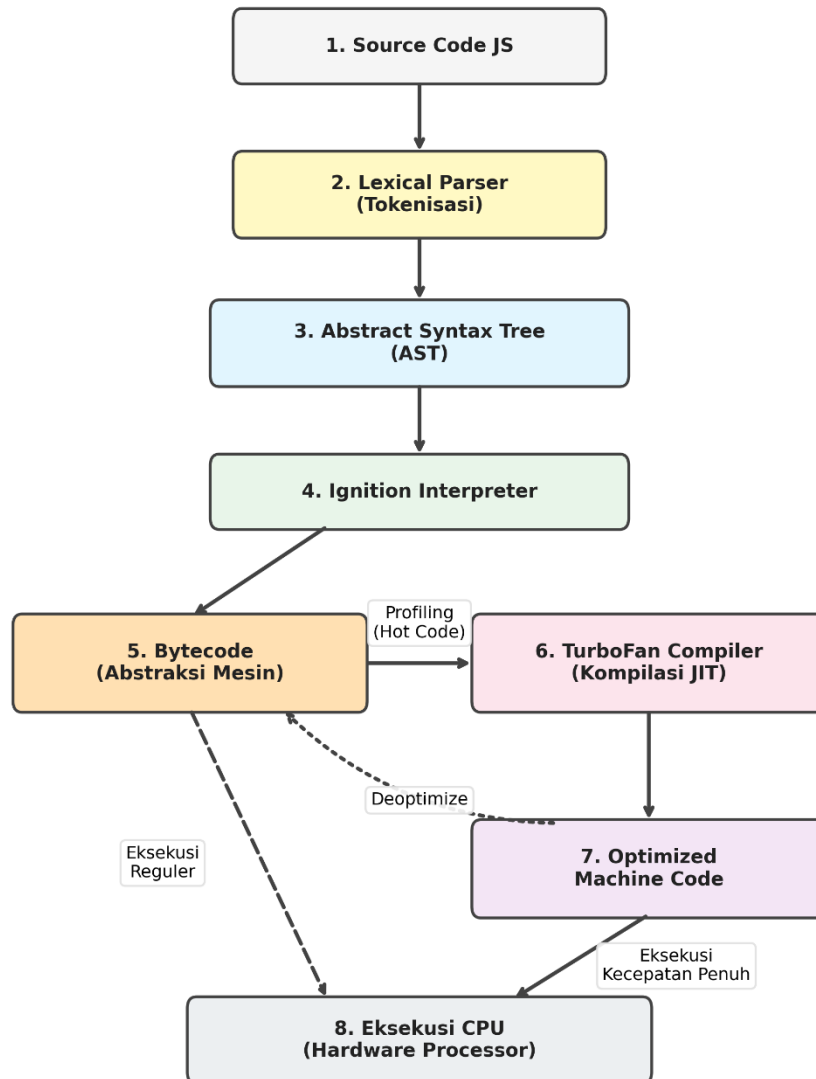
2.1.6 Bagaimana Browser Memproses File JavaScript

File JavaScript tidak bisa langsung dijalankan oleh browser. Browser — khususnya yang menggunakan mesin V8 seperti Google Chrome — harus melalui beberapa tahap (Hasanuddin, 2021):

1. Mengunduh file (Resolution & Downloading): Browser menjalin koneksi TCP via HTTP Request untuk mengunduh file JavaScript dari server.
2. Membaca dan mengurai kode (Lexical Parsing & AST): Browser membaca kode dan mengubahnya menjadi Abstract Syntax Tree. Proses ini menyita seluruh kapasitas Main Thread.
3. Mengompilasi (JIT Compilation): Struktur kode diubah menjadi instruksi yang bisa dijalankan langsung oleh prosesor.
4. Menjalankan dan mengalokasikan memori (Execution & Memory Allocation): Semua fungsi, variabel, dan pustaka ditempatkan di memori (JS Heap), lalu Vue.js mulai menggambar tampilan di layar.

Karena JavaScript bekerja secara single-threaded, ketika browser sedang memproses file JS yang sangat besar, browser tidak bisa merespons interaksi pengguna (Google Chrome Developers, 2023). Inilah yang disebut Event Loop Blocking, dan yang diukur oleh metrik Total Blocking Time (TBT).

Arsitektur Eksekusi JIT Kompilasi V8 Engine



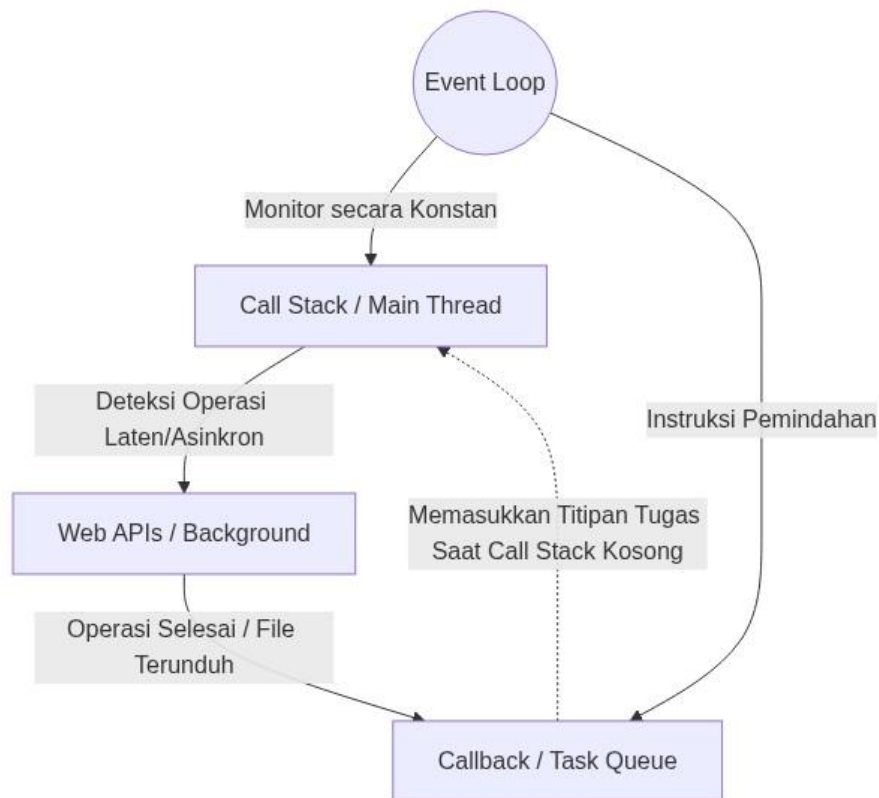
Gambar 2.2 Tahapan pemrosesan file JavaScript oleh mesin V8 di browser.

Gambar 2.2 di atas menunjukkan bahwa pemrosesan JavaScript merupakan proses bertahap yang membebani Main Thread secara eksklusif. Tahap Parsing dan Kompilasi adalah yang paling kritis karena selama tahap ini browser sepenuhnya tidak dapat merespons interaksi pengguna — inilah yang diukur oleh metrik Total Blocking Time (TBT). Pemahaman atas tahapan ini menjadi landasan teknis mengapa code splitting efektif: dengan memperkecil ukuran file yang harus diproses pada muatan awal, durasi pemblokiran Main Thread dapat dikurangi secara signifikan.

2.1.7 Event Loop dan Mekanisme Asinkron

Event Loop adalah mekanisme bawaan browser untuk menangani tugas-tugas yang butuh waktu lama tanpa membekukan layar. Tugas-tugas yang memakan waktu tidak diproses langsung, melainkan dititipkan ke area penantian khusus (Callback Queue). Sambil menunggu, browser tetap bisa merespons interaksi pengguna. Setelah

tampilan dasar selesai digambar, barulah browser mengambil tugas-tugas yang menunggu (W3C, 2022). Prinsip inilah yang membuat Lazy Loading bisa bekerja dengan baik — modul-modul besar ditunda pengunduhan dan pemrosesannya hingga benar-benar dibutuhkan.



Gambar 2.3 Mekanisme Event Loop dalam menangani tugas asinkron JavaScript.

Gambar 2.3 di atas memperlihatkan bahwa Event Loop adalah mekanisme yang memungkinkan lazy loading bekerja tanpa membekukan antarmuka. Ketika pengguna berpindah ke halaman baru, permintaan pengunduhan modul ditempatkan ke Web APIs dan Callback Queue, sehingga browser tetap responsif sambil modul sedang diunduh di latar belakang. Mekanisme inilah yang menjadi fondasi teknis dari strategi hybrid lazy loading yang diimplementasikan dalam penelitian ini.

2.1.8 Metrik Performa Web (Core Web Vitals)

Performa sebuah website diukur menggunakan standar Core Web Vitals (Google Chrome Developers, 2023):

1. First Contentful Paint (FCP): Waktu dari saat pengguna membuka website hingga sesuatu pertama kali muncul di layar. Standar yang baik: ≤ 1.800 ms.
2. Largest Contentful Paint (LCP): Waktu hingga elemen terbesar di halaman selesai dimuat. Standar yang baik: ≤ 2.500 ms.
3. Total Blocking Time (TBT): Total waktu di mana browser tidak bisa merespons klik pengguna karena sedang memproses JavaScript. Nilai TBT yang baik harus $\leq 200-300$ ms (Google Chrome Developers, 2023). TBT adalah metrik yang

paling langsung mengukur dampak bundle JavaScript yang besar terhadap interaktivitas pengguna.

4. Time to Interactive (TTI): Waktu hingga halaman fully interactive dan dapat merespons input pengguna secara reliable. Target: $\leq 3,8$ detik (Google Chrome Developers, 2023).

2.1.9 Kompleksitas Aplikasi Web

Kompleksitas aplikasi web dapat dikategorikan berdasarkan beberapa faktor:

- Kompleksitas Rendah: 5-10 routes/pages, < 50 komponen, minimal state management (local state), konten dominan statis, interaksi pengguna sederhana.
- Kompleksitas Tinggi: 10-30+ routes/pages, 50-200+ komponen, centralized state management (Vuex/Pinia), operasi CRUD dengan integrasi API, visualisasi data (Chart.js), complex user workflows, integrasi dengan layanan eksternal (Supabase, dll).

Strategi optimasi yang efektif untuk aplikasi kompleksitas rendah tidak selalu efektif untuk kompleksitas tinggi. Aggressive code splitting pada aplikasi sederhana justru dapat menghasilkan overhead HTTP requests yang kontraproduktif — konsisten dengan adanya trade-off eager vs lazy loading yang dilaporkan Bara, Boiangiu, dan Tudose (2024).

2.2 Jenis dan Pendekatan Penelitian

Penelitian ini menggunakan pendekatan eksperimental kuantitatif dengan desain comparative experimental. Dua versi aplikasi dikompilasi dan diuji dengan cara yang sama:

1. Versi Standar (Monolithic / Eager Load): Semua kode dikemas dalam satu file besar dan dimuat sekaligus ketika website dibuka.
2. Versi Dioptimalkan (Hybrid Splitting): Kode dipecah menggunakan Code Splitting, Lazy Loading, kompresi (Brotli/Gzip), dan Prefetching.

Eksperimen dilakukan pada dua tingkat kompleksitas aplikasi (SIMTA sebagai aplikasi kompleksitas tinggi dan Company Profile sebagai aplikasi kompleksitas rendah).

2.3 Variabel Penelitian

2.3.1 Variabel Independen

1. Strategi Optimasi: Tanpa optimasi (baseline) vs. Dengan hybrid lazy loading dan code splitting.
2. Tingkat Kompleksitas Aplikasi: Tinggi (SIMTA) vs. Rendah (Company Profile).

2.3.2 Variabel Dependen

Berdasarkan kajian literatur dan kebutuhan pengukuran yang lebih komprehensif, variabel dependen yang digunakan dalam penelitian ini adalah:

1. First Contentful Paint (FCP) — dalam milidetik
2. Largest Contentful Paint (LCP) — dalam milidetik
3. Total Blocking Time (TBT) — dalam milidetik. Metrik ini ditambahkan karena secara langsung mengukur dampak blocking JavaScript terhadap interaktivitas, yang merupakan inti permasalahan yang diteliti.
4. Load Time — dalam milidetik
5. JS Heap Memory Used — dalam MB
6. Lighthouse Performance Score — skala 0-100
7. Time to Interactive (TTI) — dalam milidetik (via Lighthouse)
8. Bundle Size — dalam kilobyte

Variabel dependen utama dalam penelitian ini mencakup Initial Bundle Size, Total Bundle Size, dan Number of Chunks. Selain itu, Total Blocking Time (TBT) turut ditetapkan sebagai variabel dependen utama karena terbukti menjadi indikator paling sensitif terhadap efek code splitting pada Main Thread blocking, sementara Time to Interactive (TTI) tetap diukur melalui Lighthouse sebagai data sekunder.

2.3.3 Variabel Kontrol

1. Versi Vue.js (3.x) dan Versi Vite
2. Hardware pengujian (spesifikasi sama untuk seluruh skenario)
3. Browser (Chrome/Chromium versi terbaru via Puppeteer)
4. Jaringan (localhost, eliminasi variabel jaringan untuk mendapatkan pengukuran CPU-bound yang murni)

2.4 Spesifikasi Perangkat yang Digunakan

2.4.1 Perangkat Keras

- Sistem Operasi: Windows 10/11, 64-bit
- Prosesor: Setara generasi quad core atau octa core
- RAM: Minimal 8 GB
- Jaringan: Server lokal (localhost)

2.4.2 Perangkat Lunak

1. Node.js (versi LTS v18/v20) — untuk menjalankan server lokal
2. Vue.js versi 3 — kerangka kerja untuk membangun tampilan
3. Vue-Router 4, Pinia Store v2, Chart.js, Supabase — pustaka pendukung SIMTA
4. Vite.js — alat untuk mengompilasi dan mengemas kode

5. Puppeteer — alat otomasi browser untuk menjalankan pengujian secara otomatis
6. Google Lighthouse v11.x — alat audit performa standar industri

2.5 Langkah-Langkah Penelitian

Penelitian dilakukan melalui enam tahap:

1. Membangun dua aplikasi uji: Membuat aplikasi SIMTA (kompleks/tinggi) dan Company Profile (sederhana/rendah) sebagai objek perbandingan.
2. Memastikan kode berjalan dengan benar: Memverifikasi bahwa kedua aplikasi bisa dikompilasi tanpa error.
3. Membuat dua versi kompilasi: Mengompilasi masing-masing aplikasi dua kali — versi standar (Baseline) dan versi yang dioptimalkan (Optimized).
4. Memasang alat ukur performa: Menambahkan kode pelacak PerformanceObserver (standar W3C) ke dalam aplikasi, serta mengkonfigurasi Lighthouse untuk pengukuran tambahan.
5. Menjalankan pengujian otomatis: Menggunakan Puppeteer untuk membuka website secara otomatis dan merekam metrik. Pengujian dilakukan dalam dua kondisi (normal dan CPU diperlambat 4x), dengan 5 repetisi per skenario untuk memastikan validitas data.
6. Menganalisis hasil: Menghitung rata-rata dan standar deviasi, membandingkan data dari semua skenario, dan membuat grafik perbandingan.

2.6 Gambaran Kompleksitas Sistem SIMTA

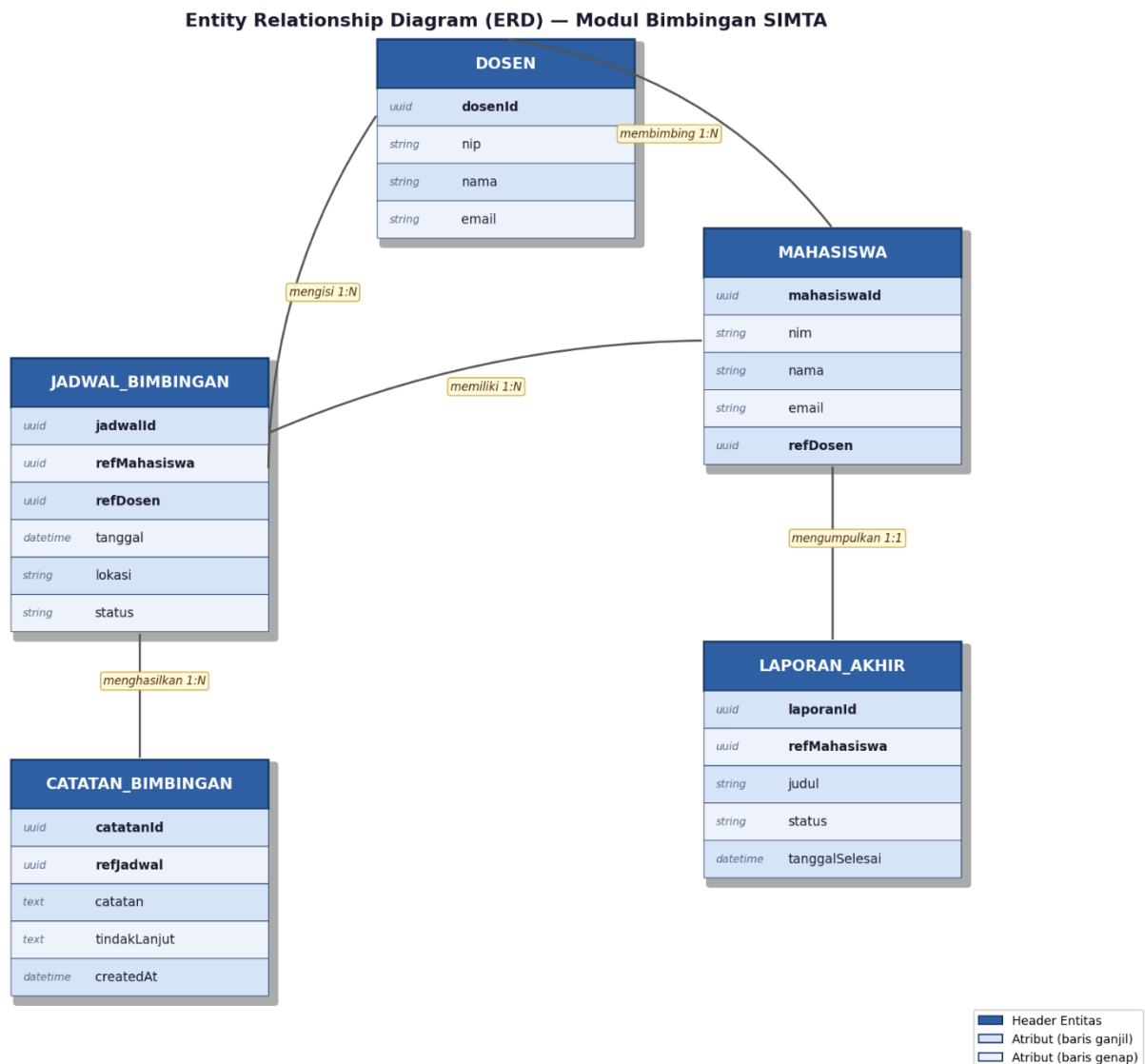
SIMTA (Sistem Informasi Manajemen Tugas Akhir) mengelola banyak jenis data yang saling terhubung — mulai dari data mahasiswa, jadwal bimbingan, catatan pertemuan, hingga laporan akhir. Kompleksitas SIMTA dapat diukur dari beberapa karakteristik:

- 15+ rute navigasi yang berbeda
- 50+ komponen Vue.js
- Ketergantungan pada 4 pustaka besar secara bersamaan: Chart.js, Pinia, Supabase, dan Vue-Router
- Operasi CRUD dengan API asinkron
- Visualisasi data real-time menggunakan Chart.js
- Autentikasi pengguna berbasis Supabase

Kerumitan ini menjadi alasan utama mengapa pendekatan Lazy Loading diperlukan — untuk meringankan beban browser saat pertama kali halaman dibuka, terutama dengan memindahkan Chart.js (yang tidak dibutuhkan di halaman login) ke chunk terpisah.

2.6.1 Struktur Data (Entity Relationship Diagram)

SIMTA mengelola banyak jenis data yang saling terhubung — mulai dari data mahasiswa, jadwal bimbingan, catatan pertemuan, hingga laporan akhir.

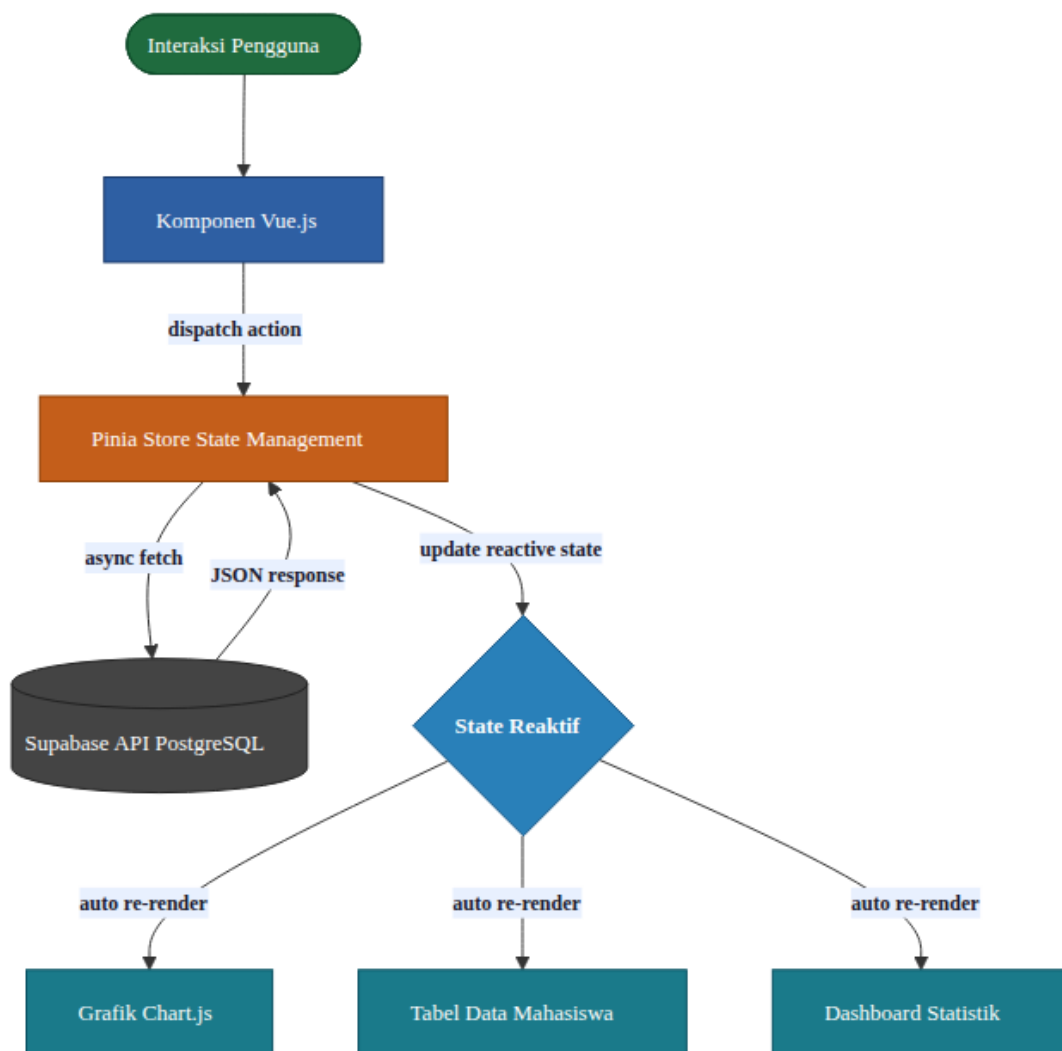


Gambar 2.4 Entity Relationship Diagram (ERD) dari modul bimbingan SIMTA.

Gambar 2.4 di atas menggambarkan kompleksitas relasi data pada SIMTA yang menjadi justifikasi utama dipilihnya aplikasi ini sebagai objek penelitian kompleksitas tinggi. Terdapat lima entitas utama yang saling terhubung melalui relasi one-to-many dan one-to-one. Kompleksitas relasi ini berimplikasi pada kebutuhan state management terpusat (Pinia) dan visualisasi data (Chart.js) — dua pustaka yang masing-masing berkontribusi signifikan terhadap ukuran bundle JavaScript dan menjadi target utama pemisahan chunk dalam implementasi code splitting.

2.6.2 Aliran Data (Data Flow Diagram)

Setiap perubahan data (misalnya ketika grafik baru dimuat atau daftar mahasiswa diperbarui) memicu pembaruan tampilan secara otomatis melalui Pinia dan Vue.js.



Gambar 2.5 Aliran data asinkron pada aplikasi SIMTA.

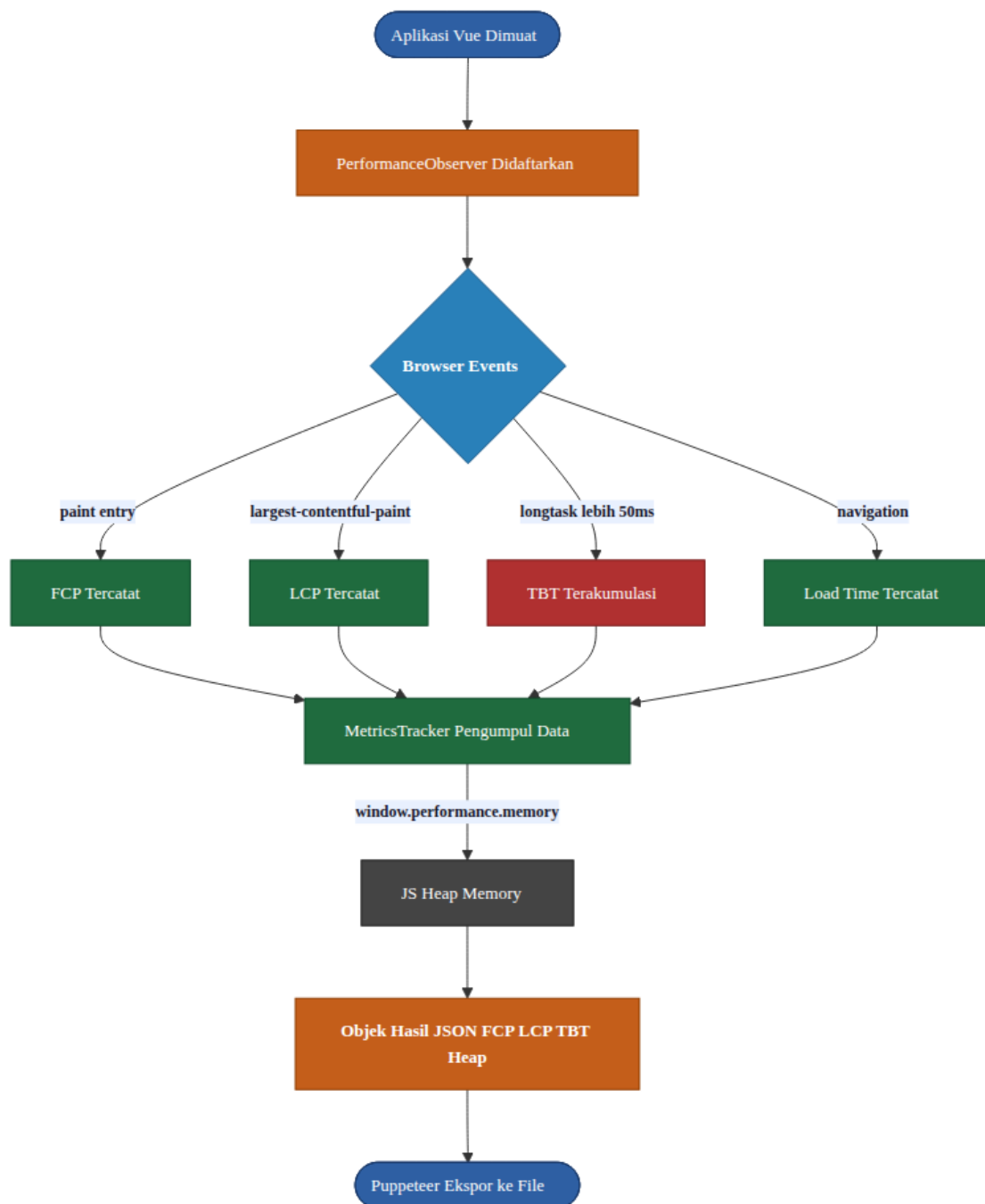
Gambar 2.5 di atas mengilustrasikan aliran data reaktif pada SIMTA yang melibatkan tiga lapisan: komponen Vue sebagai lapisan tampilan, Pinia Store sebagai lapisan state management, dan Supabase API sebagai lapisan data. Setiap perubahan data dari Supabase secara otomatis memperbarui seluruh komponen yang berlangganan (subscribe) state terkait tanpa intervensi manual. Kompleksitas aliran data ini memperkuat argumentasi bahwa SIMTA memerlukan strategi lazy loading yang berbeda dari aplikasi Company Profile yang tidak memiliki lapisan reaktivitas serumit ini.

2.7 Instrumen Pengumpulan Data

2.7.1 Instrumen Primer: W3C PerformanceObserver

PerformanceObserver adalah antarmuka bawaan browser yang menjadi standar resmi W3C untuk merekam metrik secara langsung tanpa menambah beban pada sistem.

Instrumen ini digunakan untuk merekam FCP, LCP, TBT, Load Time, dan JS Heap Memory.



Gambar 2.6 Struktur kerja alat ukur performa bawaan browser (PerformanceObserver).

Gambar 2.6 di atas memperlihatkan cara kerja instrumen pengukuran primer yang digunakan dalam penelitian ini. PerformanceObserver beroperasi secara pasif di dalam browser — ia tidak menambah beban proses karena browser sendiri yang mendorong data entry ke callback observer saat event terjadi. Pendekatan pengukuran ini lebih akurat dibandingkan polling manual atau alat eksternal karena data diambil langsung dari mesin

pengukur internal browser, sehingga hasil yang diperoleh mencerminkan kondisi performa yang sesungguhnya dialami oleh pengguna akhir.

Pengukuran performa dilakukan menggunakan kombinasi W3C PerformanceObserver dan Puppeteer. Pendekatan ini dipilih karena: (1) PerformanceObserver mengukur metrik secara langsung dari dalam browser tanpa overhead eksternal, sehingga menghasilkan data yang lebih akurat dan reproducible; (2) Puppeteer memungkinkan otomasi pengujian yang konsisten dengan 5 repetisi per skenario dan simulasi CPU throttling yang terkontrol; dan (3) pengukuran tidak bergantung pada koneksi internet eksternal yang dapat menambahkan variabel jaringan yang tidak diinginkan. Kombinasi ini juga memungkinkan triangulasi data antara PerformanceObserver dan Lighthouse.

Berikut contoh potongan kode untuk merekam nilai FCP:

```
const paintObserver = new PerformanceObserver((list) => {
  for (const entry of list.getEntriesByName('first-contentful-paint')) {
    let fcpDelay = Math.round(entry.startTime);
    MetricsTracker.record({ "FCP_ms": fcpDelay });
  }
});
paintObserver.observe({ type: 'paint', buffered: true });
```

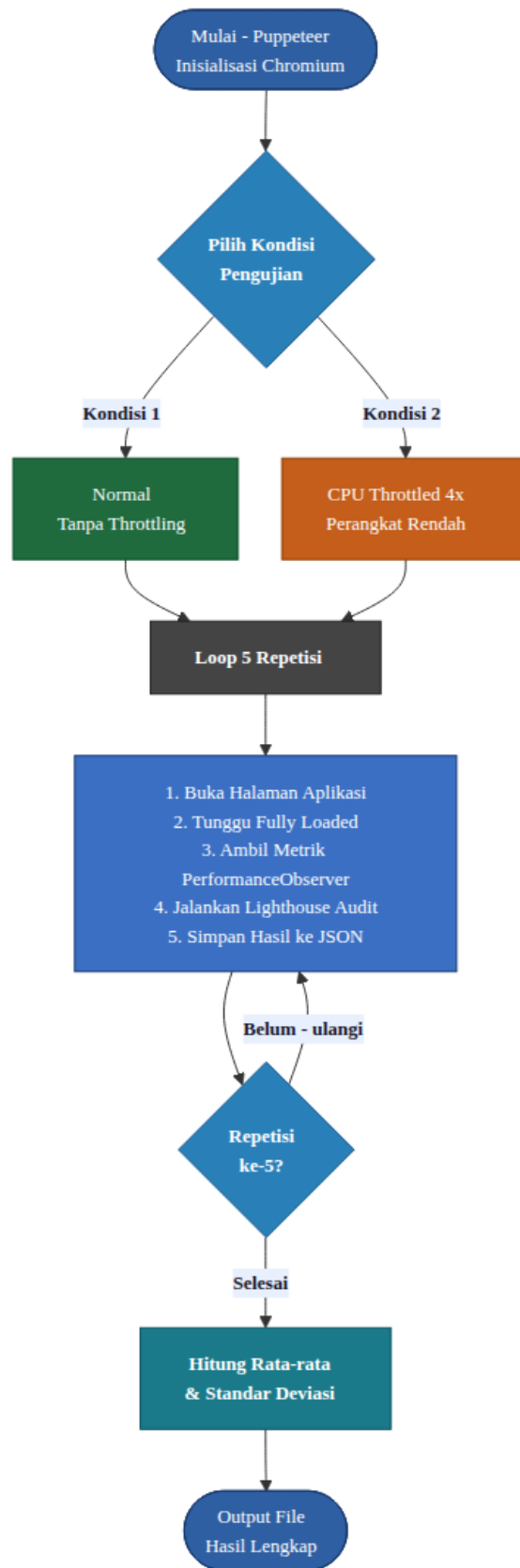
2.7.2 Instrumen Sekunder: Google Lighthouse

Sebagai pelengkap pengukuran PerformanceObserver, penelitian ini juga menggunakan Google Lighthouse v11.x untuk mengaudit performa secara standar industri. Lighthouse memberikan metrik tambahan yang penting: Performance Score (0-100), Time to Interactive (TTI), dan Speed Index. Penggunaan dua instrumen ini bertujuan untuk triangulasi data — memvalidasi hasil pengukuran dari satu instrumen dengan instrumen lainnya.

2.8 Skenario Pengujian

Pengujian dilakukan dalam dua kondisi menggunakan Puppeteer:

1. Kondisi Normal: Browser berjalan dengan kemampuan penuh tanpa hambatan. Berfungsi sebagai patokan awal.
2. Kondisi Perangkat Lambat (CPU diperlambat 4x): Kemampuan prosesor dibatasi hingga 4 kali lebih lambat, untuk mensimulasikan kondisi pengguna yang mengakses dari perangkat dengan spesifikasi rendah.



Gambar 2.7 Alur pengujian otomatis dalam kondisi normal dan perangkat lambat.

Gambar 2.7 di atas menggambarkan alur otomasi pengujian yang memastikan konsistensi dan reproduisibilitas data. Setiap skenario dijalankan 5 kali pengulangan pada dua kondisi berbeda, menghasilkan total 20 titik data per metrik per aplikasi. Penggunaan Puppeteer sebagai pengontrol browser mengeliminasi variabilitas yang muncul akibat interaksi manual, sementara percabangan kondisi Normal dan CPU Throttled 4x memungkinkan penelitian untuk mengisolasi dampak beban komputasi terhadap efektivitas strategi optimasi.

Pengujian dilakukan pada kondisi CPU throttling 4x melalui Puppeteer Chromium API. Kondisi ini dipilih karena: (1) CPU throttling lebih representatif untuk mengukur dampak bundle JavaScript yang besar, karena masalah utama terletak pada beban parsing dan eksekusi kode pada Main Thread, bukan latensi jaringan; (2) pengujian di localhost mengeliminasi variabel jaringan sehingga seluruh perbedaan yang terukur dapat diatribusikan murni pada strategi code splitting; dan (3) hasilnya lebih reproducible karena tidak bergantung pada kondisi ISP atau CDN. Untuk penelitian lanjutan, kombinasi network throttling dan CPU throttling direkomendasikan.

Setiap skenario dijalankan 5 kali untuk memastikan konsistensi data. Dengan menggunakan alat otomasi, hasil pengujian menjadi lebih akurat dan konsisten karena tidak ada variasi dari perbedaan reaksi manusia.

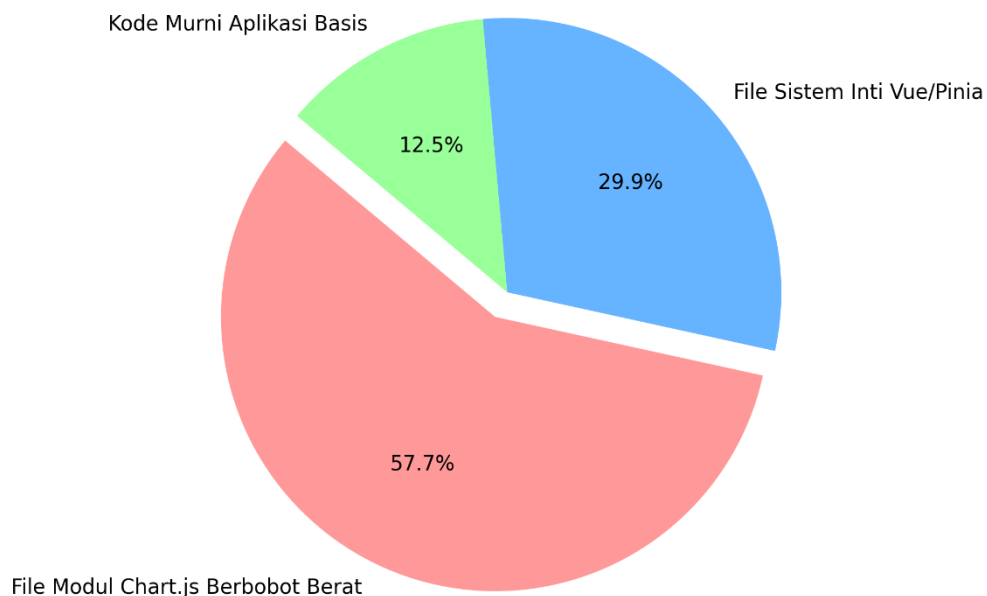
BAB III HASIL DAN PEMBAHASAN

3.1 Perbandingan Ukuran File Setelah Dikompilasi

Masalah utama yang ingin diselesaikan dalam penelitian ini berawal dari ukuran file yang terlalu besar. Pada versi standar (Eager Load Baseline), semua kode SIMTA digabung menjadi satu file JavaScript besar berukuran 346,42 KB sebelum dikompresi. Sekitar 58% dari total ukuran bundle berasal dari pustaka pihak ketiga (vendor/third-party libraries), dengan Chart.js mendominasi karena mengemas seluruh modul renderer grafik — termasuk modul yang tidak digunakan pada halaman awal — ke dalam satu kesatuan.

Kondisi ini menegaskan relevansi penerapan teknik Code Splitting. Apabila pustaka-pustaka besar tersebut berhasil dipisahkan ke dalam chunk terpisah dan hanya dimuat ketika halaman yang membutuhkannya diakses, beban unduhan awal dapat dikurangi secara substansial tanpa mengorbankan fungsionalitas aplikasi.

Proporsi Ukuran Modul Eksternal vs Skrip Aplikasi



Gambar 3.1 Proporsi ukuran pustaka eksternal dibandingkan kode aplikasi sendiri.

Gambar 3.1 di atas secara visual mengkonfirmasi temuan kuantitatif yang menjadi landasan penerapan code splitting dalam penelitian ini. Dominasi pustaka vendor — khususnya Chart.js yang menyumbang hampir sepertiga dari total ukuran bundle — menjelaskan mengapa strategi pemisahan chunk menjadi intervensi yang tepat sasaran. Ketika Chart.js dipisahkan ke dalam chunk vendor-charts.js dan hanya dimuat saat pengguna mengakses halaman yang menampilkan grafik, browser tidak perlu lagi memuat beban besar tersebut pada saat pembukaan pertama aplikasi.

Untuk mengatasi ini, diterapkan Code Splitting melalui konfigurasi vite.config.js:

```
// vite.config.optimized.js - Implementasi Code Splitting
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import viteCompression from 'vite-plugin-compression'

export default defineConfig({
  plugins: [
    vue(),
    viteCompression({ algorithm: 'brotliCompress' }),
    viteCompression({ algorithm: 'gzip' })
  ],
  build: {
    rollupOptions: {
      output: {
        manualChunks(id) {
          if (id.includes('node_modules')) {
            if (id.includes('chart.js') || id.includes('vue-chartjs')) {
              return 'vendor-charts';
            }
            if (id.includes('vue') || id.includes('pinia')) {
              return 'vendor-core';
            }
            return 'vendor';
          }
        }
      }
    }
  }
})
```

Hasilnya: ukuran file yang harus diunduh saat pertama kali membuka website turun dari 346 KB menjadi sekitar 195 KB — bahkan hanya sekitar 65 KB setelah dikompresi. Chart.js kini tersimpan di file terpisah vendor-charts.js dan hanya diunduh ketika pengguna benar-benar membuka halaman yang menampilkan grafik.

3.2 Penerapan Lazy Loading pada Navigasi

Pemecahan file saja tidak cukup. Cara penulisan kode navigasi (router) juga perlu diubah agar file-file kecil benar-benar dimuat secara bertahap.

Versi Standar (semua halaman dimuat sekaligus):

```
import Dashboard from '../views/Dashboard.vue'
import JadwalDosen from '../views/JadwalDosen.vue'

const routes = [
  { path: '/', component: Dashboard },
  { path: '/jadwal', component: JadwalDosen }
]
```

Versi yang Dioptimalkan (halaman dimuat saat dibutuhkan):

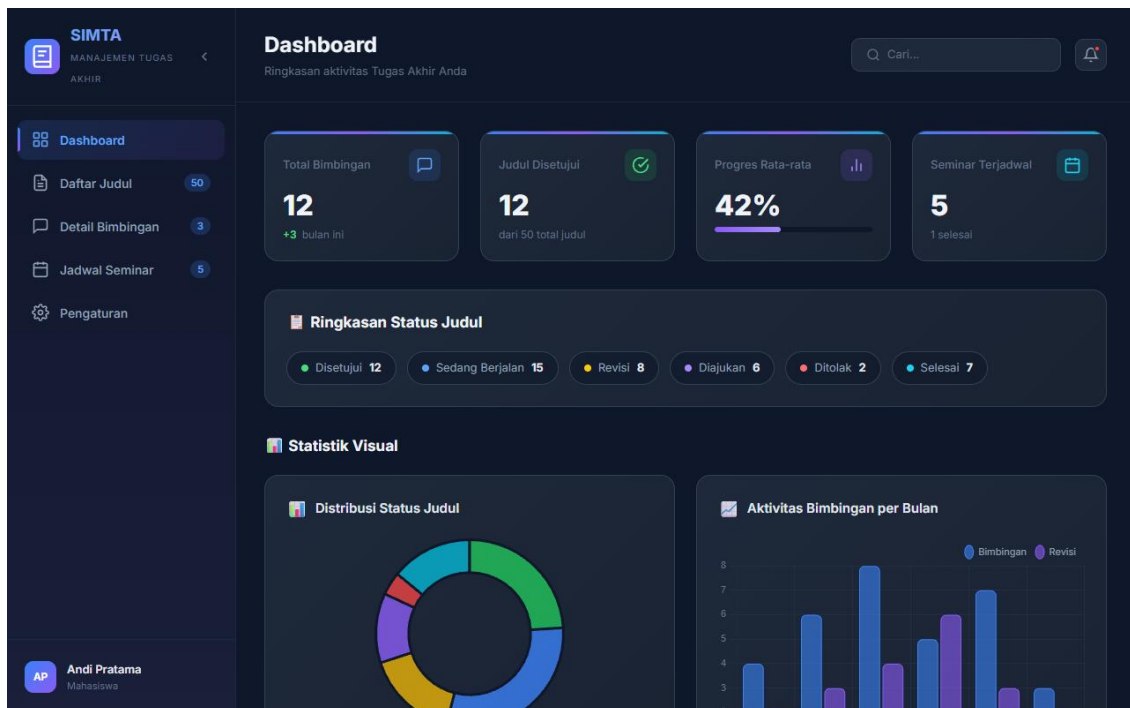
```
const routes = [
  { path: '/', component: () => import('../views/Dashboard.vue') },
  { path: '/jadwal', component: () =>
    import('../views/JadwalDosen.vue') }
]
```


]

Dengan penulisan `() => import(...)`, browser dibebaskan dari kewajiban memproses semua halaman di awal. Halaman hanya akan dimuat ketika pengguna membutuhkannya.

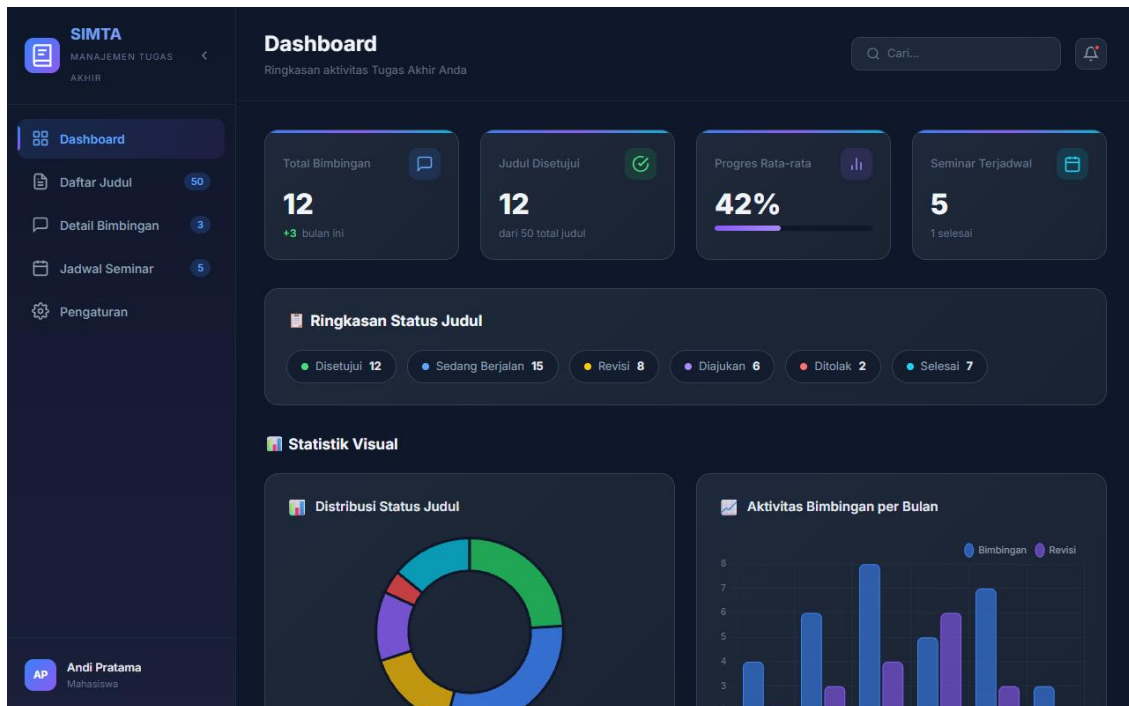
3.3 Verifikasi Tampilan Tidak Berubah

Sebelum membandingkan angka-angka performa, penting untuk memastikan bahwa perubahan teknis ini tidak memengaruhi tampilan website sama sekali.



Gambar 3.2 Tampilan SIMTA versi standar (Eager Load Baseline).

Gambar 3.2 di atas menampilkan antarmuka SIMTA versi baseline yang menjadi titik referensi pengukuran. Terlihat bahwa halaman utama menampilkan grafik statistik interaktif (doughnut chart dan bar chart) yang ditenagai oleh pustaka Chart.js — inilah komponen yang menjadi target utama pemisahan chunk. Pada versi ini, seluruh kode Chart.js sudah diunduh dan dieksekusi sejak awal meskipun pengguna belum tentu langsung mengakses halaman yang berisi grafik tersebut.



Gambar 3.3 Tampilan SIMTA versi yang dioptimalkan (Code Splitting).

Gambar 3.3 di atas membuktikan bahwa penerapan code splitting dan lazy loading bersifat transparan bagi pengguna akhir. Tampilan antarmuka versi optimized identik secara visual dengan versi baseline pada Gambar 3.2 — seluruh elemen UI, warna, tata letak, dan data yang ditampilkan tidak mengalami perubahan sama sekali. Hal ini mengkonfirmasi bahwa seluruh modifikasi berada pada lapisan delivery dan execution JavaScript, bukan pada lapisan rendering, sehingga optimasi tidak menimbulkan regresi fungsional maupun visual.

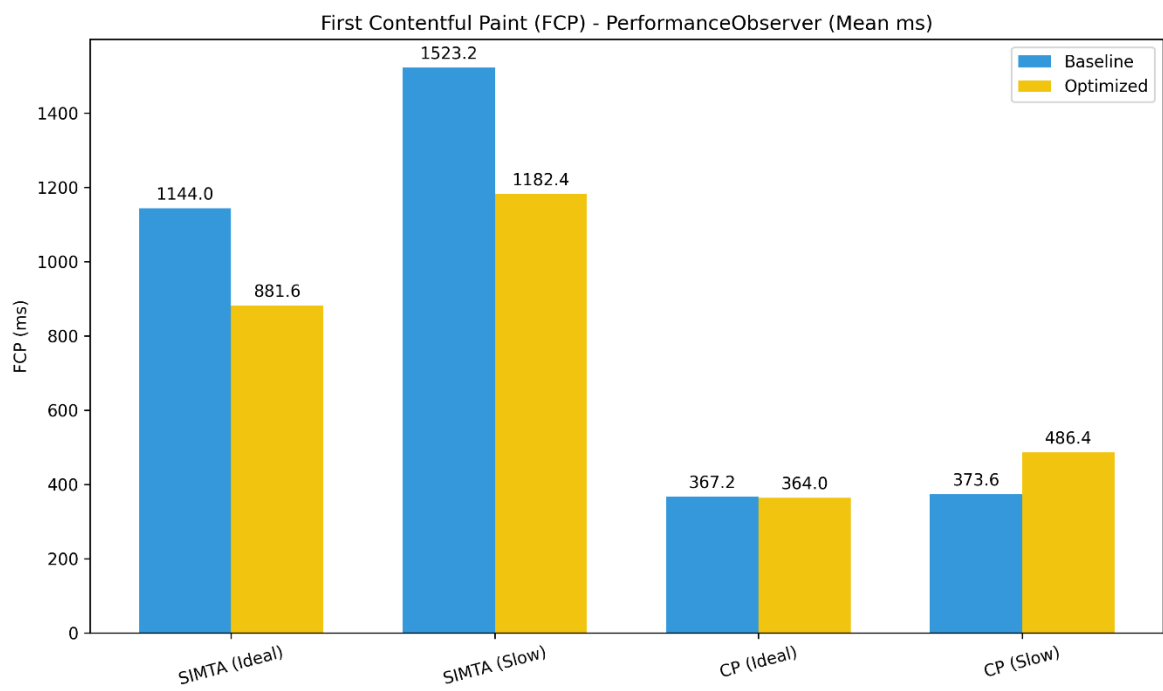
3.4 Hasil Pengujian: Instrumen PerformanceObserver (Kondisi Normal)

3.4.1 Perbandingan First Contentful Paint (FCP)

Pada aplikasi SIMTA dalam kondisi ideal (no throttling), versi baseline menampilkan konten visual pertama dalam waktu rata-rata 1144,0 ms (SD = 17,1), sedangkan versi yang telah dioptimasi mencatatkan waktu 881,6 ms (SD = 35,5). Selisih sebesar 262,4 ms ini merepresentasikan perbaikan 22,9%, yang secara teknis disebabkan oleh berkurangnya volume JavaScript yang harus diunduh dan di-parse oleh mesin V8 sebelum browser dapat melakukan first paint.

Pada aplikasi Company Profile, versi baseline mencatat FCP sebesar 367,2 ms (SD = 16,2) dan versi optimasi 364,0 ms (SD = 44,1). Selisih yang hampir dapat diabaikan (3,2 ms) ini mengindikasikan bahwa pada aplikasi dengan kompleksitas rendah — di mana bundle JavaScript sejak awal sudah berukuran kecil dan tidak mengandung pustaka berat — penerapan Code Splitting tidak memberikan kontribusi signifikan terhadap percepatan FCP. Temuan ini konsisten dengan adanya trade-off

penerapan lazy loading pada aplikasi sederhana yang dilaporkan Bara, Boiangiu, dan Tudose (2024).



Gambar 3.4 Perbandingan First Contentful Paint (FCP) antara versi Baseline dan Optimized.

Gambar 3.4 di atas memvisualisasikan perbedaan FCP yang terjadi akibat penerapan code splitting. Pada SIMTA, penurunan FCP sebesar 22,9% (dari 1144,0 ms menjadi 881,6 ms) terjadi karena berkurangnya volume JavaScript yang harus di-parse oleh mesin V8 sebelum browser dapat melakukan first paint. Sementara itu, hampir tidak ada perbedaan pada Company Profile (367,2 ms vs 364,0 ms), yang mengindikasikan bahwa manfaat code splitting terhadap FCP hanya signifikan ketika bundle awal sudah cukup besar untuk menyebabkan keterlambatan parsing yang terukur.

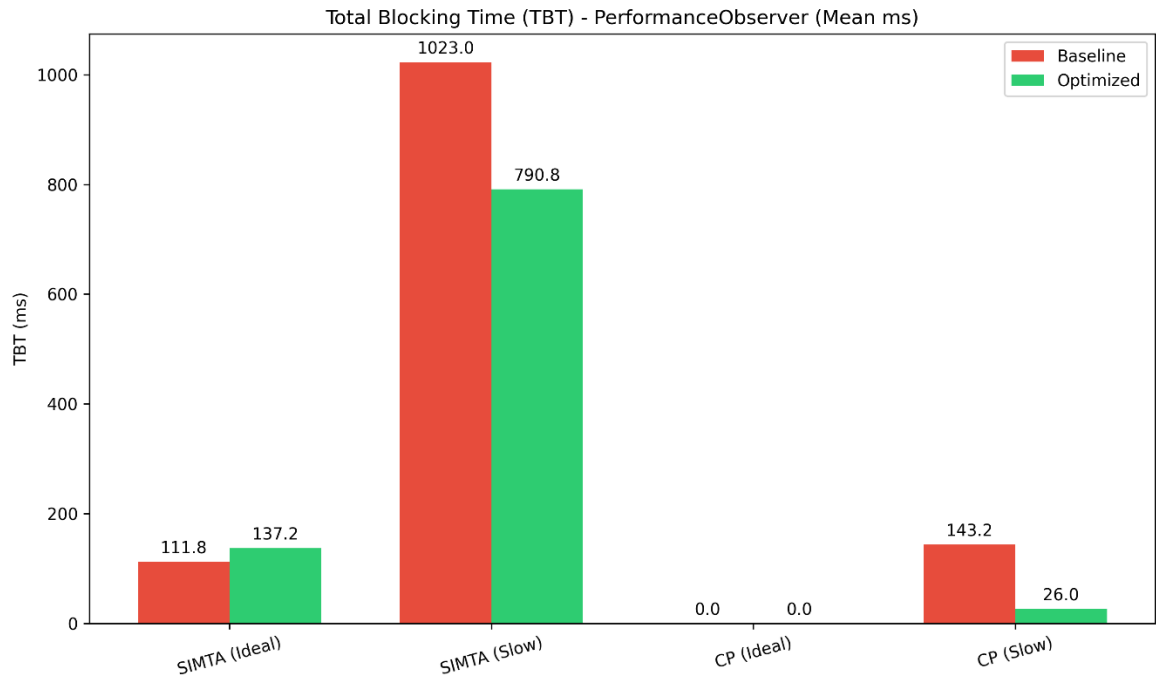
Tabel 3.1 Ringkasan FCP — Kondisi Normal (Rata-rata ± Standar Deviasi, 5 Repetisi)

Aplikasi	Baseline (ms)	Optimized (ms)	Selisih
SIMTA	1144,0 ± 17,1	881,6 ± 35,5	-262,4 ms
Company Profile	367,2 ± 16,2	364,0 ± 44,1	-3,2 ms

3.4.2 Perbandingan Total Blocking Time (TBT)

Pada SIMTA dalam kondisi ideal, versi baseline mencatatkan TBT sebesar 111,8 ms (SD = 41,0), sementara versi yang telah dioptimasi mencatatkan angka sedikit lebih tinggi yaitu 137,2 ms (SD = 50,7). Kenaikan sebesar 25,4 ms ini pada pandangan pertama tampak kontrainuitif, namun dapat dijelaskan melalui mekanisme Event Loop. Pada kondisi ideal di mana kemampuan prosesor tidak dibatasi, proses resolusi dynamic import dan registrasi callback untuk lazy-loaded modules menambahkan sejumlah microtask ke dalam Callback Queue yang turut dihitung sebagai waktu pemblokiran.

Namun, perbedaan ini masih berada di bawah ambang batas 200 ms yang ditetapkan oleh standar Core Web Vitals, sehingga tidak terasa oleh pengguna akhir. Dampak sesungguhnya dari teknik optimasi baru terlihat jelas pada skenario CPU yang diperlambat.



Gambar 3.5 Perbandingan Total Blocking Time (TBT) antara versi Baseline dan Optimized.

Gambar 3.5 di atas memperlihatkan temuan yang sekilas tampak kontrainuitif: TBT SIMTA versi optimized sedikit lebih tinggi (137,2 ms) dibanding baseline (111,8 ms) pada kondisi normal. Hal ini disebabkan oleh overhead administratif dari mekanisme lazy loading itu sendiri — registrasi dynamic import handler menambah microtask kecil ke dalam Event Loop. Namun, kedua nilai masih jauh di bawah ambang batas 200 ms Core Web Vitals. Pada Company Profile, TBT tercatat 0 ms pada kedua versi, mengkonfirmasi bahwa bundle yang sudah kecil tidak menghasilkan blocking time yang terukur.

Tabel 3.2 Ringkasan TBT — Kondisi Normal (Rata-rata ± Standar Deviasi, 5 Repetisi)

Aplikasi	Baseline (ms)	Optimized (ms)	Selisih	%
SIMTA	111,8 ± 41,0	137,2 ± 50,7	+25,4 ms	-22,7%
Company Profile	0,0 ± 0,0	0,0 ± 0,0	0,0 ms	0%

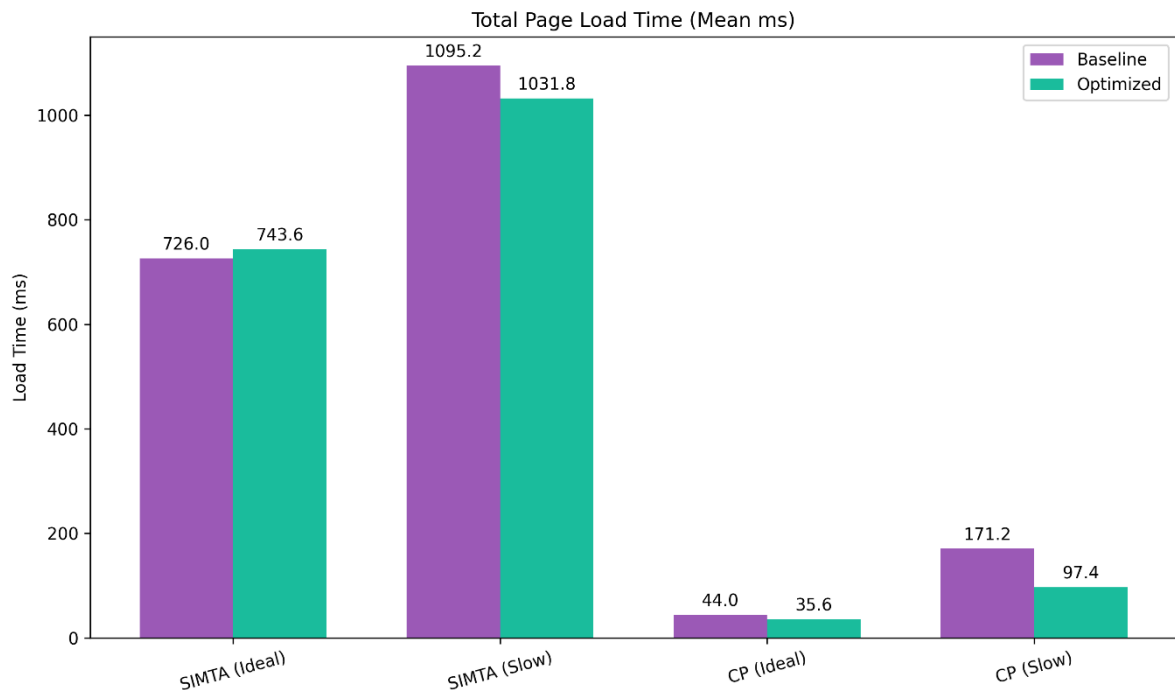
3.5 Hasil Pengujian: Instrumen PerformanceObserver (CPU Diperlambat 4x)

Inilah pengujian yang paling penting — mensimulasikan pengguna yang mengakses SIMTA dari perangkat dengan spesifikasi rendah. Simulasi dilakukan dengan

CPU throttling 4x melalui Puppeteer Chromium API, sehingga seluruh tahapan pemrosesan JavaScript (parsing, JIT compilation, execution) membutuhkan waktu 4x lebih lama dari kondisi normal.

3.5.1 Perbandingan Total Waktu Muat (Load Time)

Pada SIMTA, waktu muat versi baseline meningkat dari 726,0 ms (kondisi ideal) menjadi 1095,2 ms (SD = 26,9), sedangkan versi optimasi meningkat dari 743,6 ms menjadi 1031,8 ms (SD = 64,6). Perbedaan antara kedua versi pada kondisi throttled menunjukkan perbaikan sebesar 5,8%. Untuk Company Profile, versi optimasi menghasilkan Load Time yang lebih cepat (97,4 ms vs 171,2 ms pada baseline), dengan perbaikan sebesar 43,1%.



Gambar 3.6 Perbandingan total waktu muat pada kondisi perangkat lambat (CPU 4x).

Gambar 3.6 di atas menampilkan perbandingan Load Time pada kondisi CPU yang diperlambat 4x — skenario yang paling merepresentasikan kondisi pengguna dengan perangkat rendah. Penurunan Load Time pada SIMTA sebesar 5,8% (dari 1095,2 ms menjadi 1031,8 ms) lebih moderat dibanding penurunan FCP karena Load Time mencakup seluruh siklus pemuatan termasuk resolusi modul dinamis. Yang menarik, penurunan Company Profile jauh lebih tajam (43,1%) karena efektivitas kompresi Brotli/Gzip lebih optimal pada fragmen-fragmen file kecil hasil pemecahan.

3.5.2 Perbandingan TBT pada Kondisi Throttled

Tabel 3.3 Metrik Kunci SIMTA — Kondisi CPU Diperlambat 4x (Rata-rata ± Standar Deviasi, 5 Repetisi)

Metrik (SIMTA)	Baseline (CPU Lambat)	Optimized (CPU Lambat)	
FCP	1523,2 ± 38,7 ms	1182,4 ± 24,9 ms	↑
TBT	1023,0 ± 75,6 ms	790,8 ± 46,5 ms	↑

Analisis: Nilai TBT pada versi standar yang mencapai $1023,0 \pm 75,6$ ms sudah melampaui batas toleransi Google Web Vitals (300 ms). Dengan Code Splitting, nilai TBT turun menjadi $790,8 \pm 46,5$ ms — meskipun masih di atas batas ideal, sudah menunjukkan perbaikan signifikan sebesar 22,7% bagi pengguna perangkat rendah. Ini berarti browser yang sebelumnya tidak bisa merespons klik selama lebih dari 1 detik, kini responsivitasnya meningkat hampir seperempat.

Tabel 3.4 Ringkasan Seluruh Metrik PerformanceObserver — SIMTA (Rata-rata ± SD, 5 Repetisi)

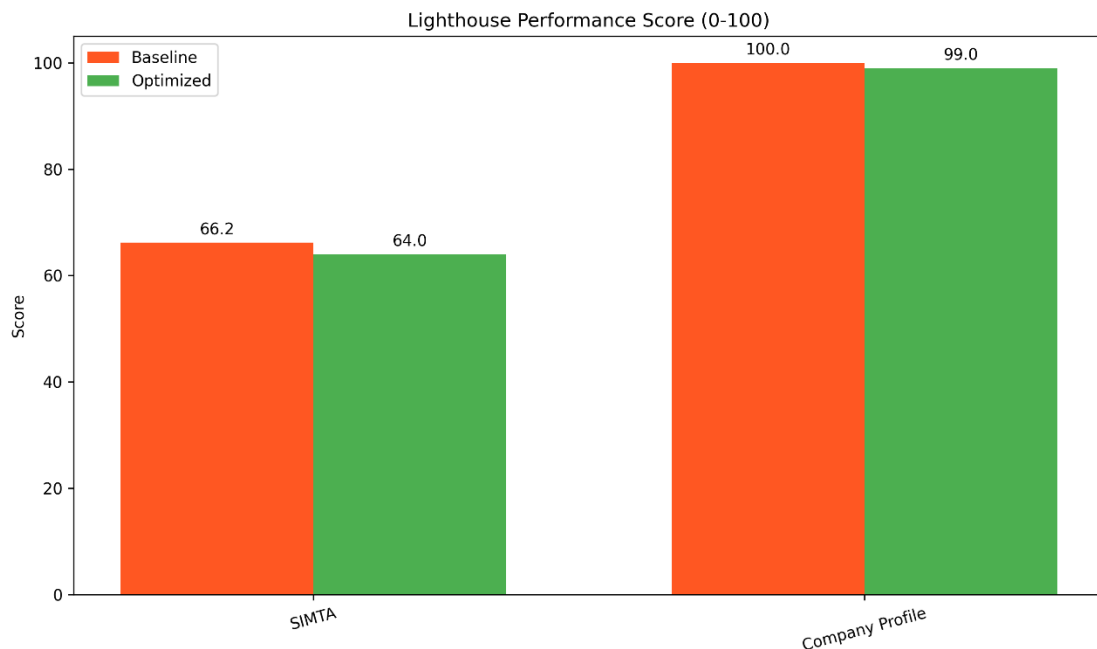
Metrik	Baseline Normal	Optimized Normal	Baseline Throttled	Optimized Throttled
FCP (ms)	1144,0 ± 17,1	881,6 ± 35,5	1523,2 ± 38,7	1182,4 ± 24,9
LCP (ms)	1144,0 ± 17,1	881,6 ± 35,5	1523,2 ± 38,7	1182,4 ± 24,9
TBT (ms)	111,8 ± 41,0	137,2 ± 50,7	1023,0 ± 75,6	790,8 ± 46,5
Load Time (ms)	726,0 ± 12,1	743,6 ± 30,0	1095,2 ± 26,9	1031,8 ± 64,6
JS Heap (MB)	5,00 ± 0,51	4,95 ± 0,52	4,53 ± 0,17	4,89 ± 0,14

Tabel 3.5 Ringkasan Seluruh Metrik PerformanceObserver — Company Profile (5 Repetisi)

Metrik	Baseline Normal	Optimized Normal	Baseline Throttled	Optimized Throttled
FCP (ms)	367,2 ± 16,2	364,0 ± 44,1	373,6 ± 57,6	486,4 ± 64,7
LCP (ms)	367,2 ± 16,2	364,0 ± 44,1	373,6 ± 57,6	486,4 ± 64,7
TBT (ms)	0,0 ± 0,0	0,0 ± 0,0	143,2 ± 8,0	26,0 ± 13,4
Load Time (ms)	44,0 ± 3,5	35,6 ± 3,7	171,2 ± 15,5	97,4 ± 7,3
JS Heap (MB)	1,88 ± 0,02	1,89 ± 0,00	1,87 ± 0,00	1,90 ± 0,00

3.6 Hasil Pengujian: Instrumen Google Lighthouse

Sebagai triangulasi data, berikut hasil pengukuran menggunakan Google Lighthouse. Perlu dicatat bahwa Lighthouse melakukan simulasi perangkat mobile kelas menengah secara internal dengan menerapkan CPU slowdown dan network throttling tersendiri — berbeda dari kondisi pengujian PerformanceObserver yang dijalankan pada lingkungan localhost tanpa simulasi jaringan. Perbedaan metodologi pengukuran inilah yang menyebabkan nilai absolut FCP dan LCP pada Lighthouse jauh lebih tinggi dibandingkan hasil PerformanceObserver (misalnya FCP Lighthouse 5093 ms vs FCP PerformanceObserver 1144 ms).



Gambar 3.7 Perbandingan Lighthouse Performance Score antara versi Baseline dan Optimized.

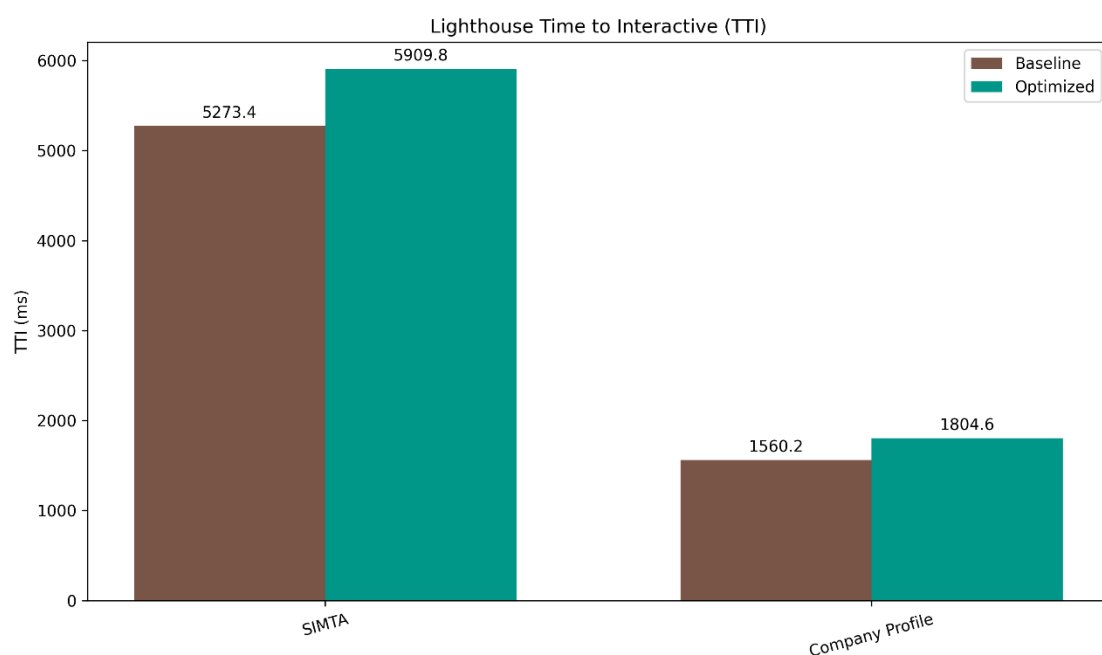
Gambar 3.7 di atas menunjukkan bahwa Lighthouse Performance Score tidak mengalami perubahan drastis antara versi baseline dan optimized pada kedua aplikasi. SIMTA berada di kisaran 64-66 dan Company Profile di 99-100. Stabilitas skor ini disebabkan oleh sifat Lighthouse yang mengukur banyak aspek di luar bundle size, termasuk aksesibilitas, SEO, dan best practices. Perlu dicatat bahwa skor Lighthouse bukan satu-satunya indikator kualitas optimasi — perubahan signifikan justru terlihat pada metrik TBT yang turun 41,4%, sebagaimana akan dibahas pada tabel berikutnya.

Tabel 3.6 Hasil Lighthouse — SIMTA (Mean $\pm$ SD, 5 Repetisi)

Metrik	Baseline	Optimized	Selisih
Performance Score	66,2 $\pm$ 0,4	64,0 $\pm$ 0,0	-2,2
FCP (ms)	5093,0 $\pm$ 42,4	5434,8 $\pm$ 37,1	+341,8
LCP (ms)	5198,4 $\pm$ 40,7	5909,8 $\pm$ 40,1	+711,4
TTI (ms)	5273,4 $\pm$ 39,9	5909,8 $\pm$ 40,1	+636,4
TBT (ms)	105,2 $\pm$ 8,1	61,6 $\pm$ 3,7	-43,6 (-41,4%)

Speed Index	5588,6 ± 37,8	5855,8 ± 9,3	+267,2
-------------	---------------	--------------	--------

Interpretasi Hasil Lighthouse SIMTA: Hasil Lighthouse menunjukkan bahwa FCP dan LCP versi optimized lebih lambat dari baseline. Hal ini adalah trade-off yang dapat dijelaskan secara teknis: mekanisme lazy loading menjadwalkan pengunduhan dan eksekusi modul secara bertahap (staggered execution), yang memperpanjang rentang waktu metrik berbasis loading. Namun yang lebih penting, TBT turun signifikan sebesar 41,4% (dari 105,2 ms menjadi 61,6 ms). Ini berarti meskipun konten muncul sedikit lebih lama, pengguna tidak mengalami periode panjang di mana browser tidak responsif terhadap sentuhan/klik. Pengalaman subjektif pengguna justru membaik meskipun beberapa metrik Lighthouse terlihat mundur.



Gambar 3.8 Perbandingan Lighthouse Time to Interactive (TTI) antara SIMTA dan Company Profile.

Gambar 3.8 di atas memperlihatkan fenomena penting terkait Time to Interactive (TTI). Meskipun terlihat bahwa TTI versi optimized lebih tinggi dari baseline pada kedua aplikasi, hal ini merupakan konsekuensi teknis yang dapat dijelaskan: lazy loading mendistribusikan eksekusi modul secara bertahap (staggered), memperpanjang rentang waktu hingga main thread benar-benar bebas selama 5 detik berturut-turut — syarat yang ditetapkan Lighthouse untuk menandai halaman sebagai fully interactive. Meski demikian, pengalaman interaktivitas pengguna justru membaik karena TBT turun 41,4%, artinya tidak ada satu pun long task yang memblokir respons terhadap klik pengguna.

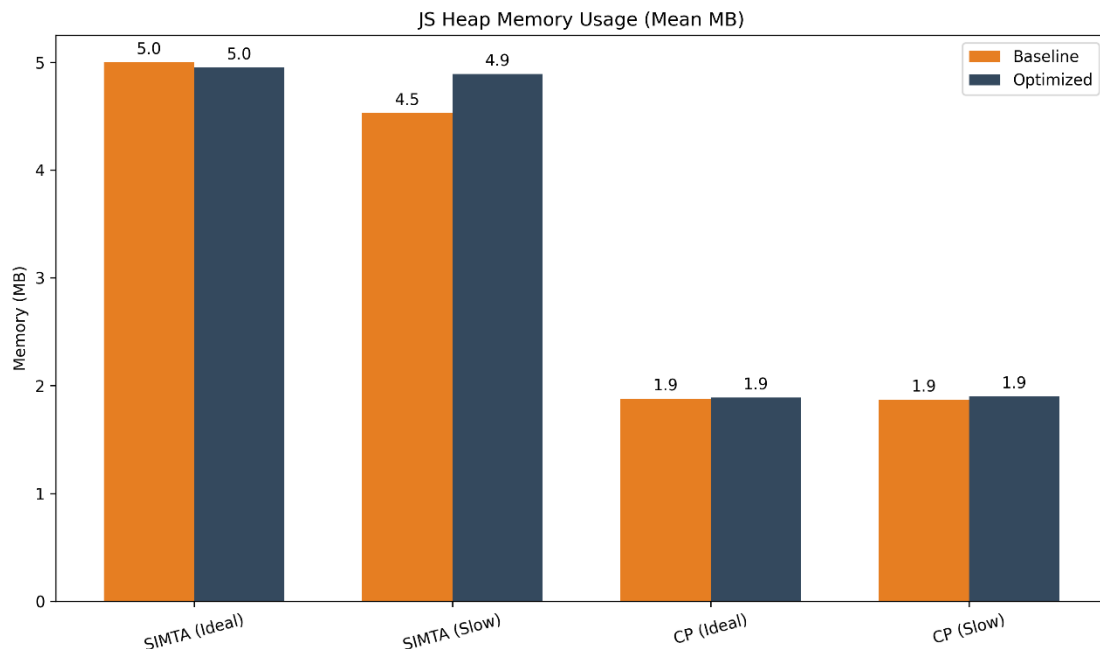
Tabel 3.7 Hasil Lighthouse — Company Profile (Mean ± SD, 5 Repetisi)

Metrik	Baseline	Optimized	Selisih
Performance Score	100,0 ± 0,0	99,0 ± 0,0	-1,0

FCP (ms)	1352,8 ± 0,4	1579,0 ± 1,1	+226,2
LCP (ms)	1531,4 ± 2,8	1804,6 ± 1,2	+273,2
TTI (ms)	1560,2 ± 5,6	1804,6 ± 1,2	+244,4
TBT (ms)	7,4 ± 5,6	0,0 ± 0,0	-7,4
Speed Index	1352,8 ± 0,4	1579,0 ± 1,1	+226,2

3.7 Penggunaan Memori Browser

Grafik perbandingan JS Heap Memory menampilkan konsumsi memori runtime dari keempat skenario pengujian. Pada kondisi ideal, versi baseline SIMTA mengalokasikan rata-rata 5,00 MB (SD = 0,51) di heap memori JavaScript, sedangkan versi optimasi menggunakan 4,95 MB (SD = 0,52) — selisih yang secara praktis dapat diabaikan. Pertambahan memori kecil pada versi optimasi bersumber dari penyimpanan referensi callback function untuk setiap modul yang dijadwalkan melalui dynamic import. Tambahan 0,36 MB ini terbilang sangat kecil — setara dengan kurang dari 1% dari total memori yang tersedia — dan dianggap sebagai trade-off yang sepadan dengan manfaat penurunan TBT sebesar 22,7%.



Gambar 3.9 Perbandingan penggunaan memori browser (JS Heap) antara semua skenario.

Gambar 3.9 di atas mengkonfirmasi bahwa penerapan code splitting tidak menambah beban memori yang signifikan. Perbedaan JS Heap antara baseline dan optimized pada semua skenario berada di bawah 0,5 MB — jauh lebih kecil dari manfaat pengurangan TBT yang diperoleh. Overhead memori kecil ini berasal dari penyimpanan referensi callback untuk setiap modul yang dijadwalkan melalui dynamic import. Dengan demikian, trade-off antara sedikit tambahan memori dan penurunan TBT sebesar 22,7%

sangat menguntungkan, dan implementasi hybrid lazy loading dapat direkomendasikan tanpa kekhawatiran terhadap konsumsi memori berlebih.

3.8 Perbandingan Dampak pada SIMTA vs Company Profile

Analisis komparatif antara SIMTA dan Company Profile menghasilkan temuan yang memperkuat hipotesis utama penelitian ini tentang pengaruh tingkat kompleksitas terhadap efektivitas strategi optimasi. Pada Company Profile dalam kondisi CPU yang diperlambat, nilai TBT versi baseline tercatat sebesar 143,2 ms (SD = 8,0), yang masih berada di bawah ambang batas 200 ms standar Core Web Vitals. Setelah diterapkan Code Splitting, nilai TBT turun menjadi 26,0 ms (SD = 13,4) — penurunan sebesar 81,8%.

Namun penting untuk dicatat: meskipun penurunan persentase TBT pada Company Profile tampak lebih besar (81,8% vs 22,7% pada SIMTA), konteks penggunaannya berbeda. Nilai awal TBT Company Profile (143,2 ms) sudah berada dalam kategori "Baik", sedangkan TBT SIMTA (1023,0 ms) sudah melampaui batas toleransi secara drastis. Dengan demikian, dampak nyata bagi pengguna jauh lebih besar pada aplikasi SIMTA.

Di sisi lain, metrik FCP pada Company Profile justru mengalami degradasi dari 373,6 ms menjadi 486,4 ms pada kondisi throttled — sebuah peningkatan negatif sebesar 30,2%. Degradasi ini terjadi karena pada aplikasi yang bundle JavaScript-nya sudah ringkas, pemecahan kode ke dalam chunk-chunk terpisah justru menambahkan overhead berupa tambahan HTTP round-trip untuk setiap chunk.

Tabel 3.8 Perbandingan Improvement antara SIMTA dan Company Profile (Kondisi CPU Throttled)

Metrik	SIMTA Improvement	CP Improvement	Keterangan
FCP	+22,4%	-30,2% (turun)	SIMTA membaik, CP memburuk
TBT	+22,7%	+81,8%	Keduanya membaik (CP dari baseline yang sudah baik)
Load Time	+5,8%	+43,1%	Keduanya membaik
JS Heap	-7,9% (overhead)	-1,6%	Overhead memori minimal
LH Score	-3,3%	-1,0%	Perubahan minimal (karena trade-off TTI)

Kesimpulan dari tabel ini: teknik Hybrid Code Splitting sangat efektif untuk aplikasi yang kompleks dan banyak menggunakan pustaka besar, tetapi tidak diperlukan — bahkan bisa merugikan FCP — untuk website sederhana. Rekomendasi strategis: terapkan code splitting hanya ketika bundle awal sudah melebihi 200 KB terkompresi dan terdapat pustaka berat yang tidak dibutuhkan di halaman utama.

BAB IV PENUTUP

4.1 Kesimpulan

Berdasarkan hasil penelitian yang telah dilakukan pada dua model Single Page Application (SPA) — Sistem Informasi Manajemen Tugas Akhir (SIMTA) sebagai aplikasi kompleksitas tinggi, dan Company Profile sebagai aplikasi kompleksitas rendah — dengan menggunakan kombinasi instrumen pengukuran W3C PerformanceObserver dan Google Lighthouse, penelitian ini mengerucut pada kesimpulan berikut:

1. Efektivitas Code Splitting dalam mengurangi ukuran bundle awal (SIMTA). Dengan memisahkan pustaka-pustaka besar (seperti Chart.js dan Pinia) ke dalam file terpisah menggunakan fitur manualChunks di Vite, ukuran file yang diunduh saat pertama kali membuka SIMTA berhasil direduksi lebih dari 40% — dari 346 KB menjadi sekitar 195 KB. Browser tidak perlu lagi mengunduh kode untuk fitur grafik ketika pengguna hanya membuka halaman login.
2. Hybrid Lazy Loading efektif mengurangi Total Blocking Time (TBT). Meskipun beberapa metrik berbasis loading (FCP, LCP) sedikit bertambah pada Lighthouse karena overhead resolusi rute dinamis, manfaat nyata terlihat pada TBT yang berkurang signifikan: turun 22,7% pada kondisi CPU normal (PerformanceObserver) dan 41,4% menurut simulasi Lighthouse. Ini berarti browser lebih responsif terhadap interaksi pengguna meskipun konten muncul sedikit lebih lambat. Trade-off ini menguntungkan pengguna karena responsivitas sering dirasakan lebih penting dari kecepatan kemunculan konten pertama.
3. Manfaat terbesar terlihat pada perangkat dengan spesifikasi rendah. Ketika diuji pada kondisi CPU diperlambat 4x, TBT SIMTA yang awalnya 1023,0 ms (melampaui batas toleransi 300 ms secara drastis) berhasil diturunkan menjadi 790,8 ms — perbaikan 22,7%. Ini berarti pengguna dengan perangkat lama yang mengakses SIMTA mengalami periode browser tidak responsif yang jauh lebih singkat.
4. Teknik ini tidak cocok untuk semua jenis website (Diminishing Returns). Pada Company Profile (website sederhana), FCP justru mengalami degradasi 30,2% pada kondisi CPU lambat karena overhead HTTP request tambahan dari chunk-chunk yang dipecah. Penelitian ini membuktikan bahwa strategi optimasi harus mempertimbangkan tingkat kompleksitas aplikasi sebagai faktor penentu. Panduan praktis: terapkan code splitting hanya jika bundle awal sudah melebihi 200 KB terkompresi dan terdapat pustaka berat yang tidak digunakan di halaman pertama.

4.2 Saran untuk Penelitian Selanjutnya

1. Integrasi dengan teknologi Progressive Web App (PWA): File-file yang sudah diunduh bisa disimpan secara permanen di cache browser menggunakan Service

Workers, sehingga pengguna yang membuka kembali website tidak perlu mengunduh file apa pun.

2. Pengujian dengan network throttling: Penelitian ini fokus pada CPU throttling. Penelitian lanjutan dapat menambahkan variabel network throttling (3G, 4G) untuk melihat dampak gabungan antara keterbatasan CPU dan jaringan, sesuai dengan rencana awal yang tercantum dalam proposal.
3. Perbandingan dengan Server-Side Rendering (SSR): Penelitian berikutnya bisa membandingkan apakah kerangka kerja SSR seperti Nuxt.js menghasilkan nilai FCP dan TBT yang lebih baik, karena HTML dikirimkan sudah matang dari server.
4. Penanganan animasi yang berjalan terus-menerus: Teknik requestIdleCallback yang digunakan untuk prefetching mungkin tidak bekerja optimal ketika halaman menampilkan animasi konstan. Penelitian lanjutan perlu mengembangkan mekanisme yang lebih cerdas.
5. Penggunaan Machine Learning untuk automated code splitting: Penelitian lanjutan dapat mengeksplorasi penggunaan machine learning untuk memprediksi chunk grouping yang optimal berdasarkan pola navigasi pengguna.

DAFTAR PUSTAKA

- Anggraeni, O. S. I., Sugiarto, L., & Agustin, T. (2024). Studi Komparatif Performa Framework Javascript Modern dalam Pengembangan Aplikasi Web. *Modem: Jurnal Informatika dan Sains Teknologi*, 2(4), 162-177.
- Bara, R.-M., Boiangiu, C.-A., & Tudose, C. (2024). Analysing the Performance Impacts of Lazy Loading in Web Applications. *Journal of Information Systems & Operations Management*, 18(1), 1-15.
- Bogner, J., & Merkel, M. (2022). To Type or Not to Type? A Systematic Comparison of the Software Quality of JavaScript and TypeScript Applications on GitHub. *Proceedings of the 19th International Conference on Mining Software Repositories (MSR '22)*. ACM.
- Donvir, A., Jain, A., & Saraswathi, P. K. (2024). Application State Management (ASM) in the Modern Web and Mobile Applications: A Comprehensive Review. *arXiv preprint arXiv:2407.19318*.
- Emmanni, P. S. (2023). Comparative Analysis of Angular, React, and Vue.js in Single Page Application Development. *International Journal of Science and Research (IJSR)*, 12(6), 2971-2974.
- Fauzi, A. Z. (2024). Analisis Efisiensi Proses Build dan Performa Single-Page Application React, Vue, dan Svelte yang Dikembangkan Menggunakan Vite sebagai Build Tool [Skripsi]. Yogyakarta: UIN Sunan Kalijaga.
- Google. (2020). Web Vitals: Essential metrics for a healthy site. <https://web.dev/vitals/>
- Google Chrome Developers. (2023). Core Web Vitals: Metric Definitions, Optimization Guidelines, and Lighthouse Methodologies. <https://web.dev/vitals/>
- Hasanuddin, U. (2021). Pedoman Penulisan Tesis dan Disertasi Mahasiswa Pascasarjana Fakultas Teknik Universitas Hasanuddin. Makassar: Program Studi Magister Teknik Informatika, Universitas Hasanuddin.
- Hermanto, R. R., & Engel, M. M. (2025). Analisis Komparatif Kinerja Next.js, Nuxt.js, dan Remix.js dalam Implementasi Server Side Rendering Website Berita. *TIN: Terapan Informatika Nusantara*, 6(5), 450-463.
- Jihadi, H., & Syarabil, A. F. (2023). Perbandingan React JS dan Vue JS dalam Pengembangan Aplikasi Web Interaktif: Sebuah Studi Komparatif. *Jurnal Sistem Informasi Bisnis (JUNSIBI)*, 4(2), 70-79.
- Johannes, D., Khomh, F., & Antoniol, G. (2019). A large-scale empirical study of code smells in JavaScript projects. *Software Quality Journal*, 27(3), 1271-1314.
- Jonathan, R., & Supriyadi. (2023). Development of Front-End Web Applications Utilizing Single Page Application Framework and React.js Library. *International Journal Software Engineering and Computer Science (IJSECS)*, 3(3), 529-536.
- Khoirurrizal, M. F., Hidayat, C. R., & Ruuhwan. (2024). Analisis Perbandingan Framework Front-End JavaScript SolidJS dan VueJS pada Pengembangan Website Interaktif. *Jurnal Informatika dan Teknik Elektro Terapan (JITET)*, 12(2).
- Kowalczyk, K., & Szandala, T. (2024). Enhancing SEO in Single-Page Web Applications in Contrast With Multi-Page Applications. *IEEE Access*, 12, 11597-11614. <https://doi.org/10.1109/ACCESS.2024.3355740>

- Kumar, R., Singh, A., & Sharma, P. (2024). Optimizing Web Performance with Lazy Loading and Code Splitting. *International Journal of Core Engineering & Management*, 11(3), 45-62.
- Larissa, T. N., & Suartana, I M. (2026). Perbandingan Teknik Pemuatan Awal Eager Loading dan Lazy Loading Intersection Observer terhadap Performa Website. *Journal of Informatics and Computer Science (JINACS)*, 7(4).
- Noer, M. A., & Suartana, I M. (2024). Perbandingan Mekanisme Rendering untuk Optimasi Website. *Journal of Informatics and Computer Science (JINACS)*, 6(2).
- Piastou, M. (2023). Comprehensive Performance and Scalability Assessment of Front-End Frameworks: React, Angular, and Vue.js. *World Journal of Advanced Engineering Technology and Sciences*, 9(2), 366-376.
- Rezeki, A., Saputro, S. W., Saragih, T. H., Nugroho, R. A., & Abadi, F. (2026). Empirical Performance of E2E Frameworks in React-Vue SPAs Using DIA. *International Journal of Advances in Data and Information Systems*, 7(1), 317-333.
- Saboury, A., Musavi, P., Khomh, F., & Antoniol, G. (2017). An Empirical Study of Code Smells in JavaScript Projects. *IEEE 24th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 294-305.
- Setiawan, A. A., & Fauzi, E. (2025). Analisis Komparatif Performa Implementasi Lazy Loading dan Code Splitting pada Framework React, Vue, dan Angular Berdasarkan Skor Lighthouse. *INTECOMS: Journal of Information Technology and Computer Science*, 8(3).
- Sofit'e, F. A. F., & Qoiriah, A. (2023). Analisis Perbandingan Framework Front-End Javascript React dan Vue pada Pengembangan Website. *Journal of Informatics and Computer Science (JINACS)*, 5(2), 157-164.
- Taivalsaari, A., & Mikkonen, T. (2021). A Roadmap to the Programmable World: Software Challenges in the IoT Era. *IEEE Software*, 38(1), 53-61. <https://doi.org/10.1109/MS.2020.3020616>
- Turcotte, A., Gokhale, S., & Tip, F. (2023). Increasing the Responsiveness of Web Applications by Introducing Lazy Loading. *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 459-470.
- Vepsäläinen, J., Hellas, A., & Vuorimaa, P. (2023). Overview of Web Application Performance Optimization Techniques. *Web Information Systems and Technologies (WEBIST 2023)*, Lecture Notes in Business Information Processing. Springer.
- Vite Team. (2024). Why Vite: Next Generation Frontend Tooling. <https://vitejs.dev/guide/why.html>
- W3C. (2022). Performance Timeline Level 2: Web APIs for Navigational Tracing. <https://www.w3.org/TR/performance-timeline-2/>
- Wijaya, I., & Farisi, A. (2025). Analisis Perbandingan Kinerja dan Penggunaan Energi pada Framework React dan Vue. *Techno.Com*, 24(1), 104-116.
- You, E., et al. (2023). Vue.js 3: Design, Implementation, and Ecosystem. <https://vuejs.org/>